Universidad Nacional Autónoma de México

Facultad de Estudios Superiores Acatlán





Database Administrator

Empresa - Proyecto

Query Teach Solutions

Movilidad: Ecobici CDMX

Integrantes

Estrada Romero Meliza Edith

Ricardo López Ramírez

Ruiz Sánchez Emiliano

Cabrera Trejo José Luis

Actualización: 4 de noviembre de 2026

Índice

1. Tema y Descripción del Proyecto **1**
- 1.1. Descripción General . . . 1
- 1.2. Elección del Tema . . . 1
- 1.3. Información a Visualizar en la Base de Datos . . . 1
- 1.4. Finalidad Principal . . . 1

2. Diagrama Entidad-Relación **2**

3. Enlace a Repositorio GitHub. **3**

4. Enlace Google Drive **3**

5. Estimación Teórica y Proyección de Espacio en Disco LocalHost. **4**
- 5.1. 1. Tamaño por tipo de dato y fila (Row Size) . . . 4
- 5.2. 2. Cálculo del Espacio de Carga Inicial . . . 4
- 5.3. 3. Simulación y Proyección de Crecimiento . . . 4

6. Consulta Diccionario de Datos y Comparación de Almacenamiento LocalHost. **5**
- 6.1. Justificación Teórica . . . 5
- 6.2. Desarrollo de Consultas SQL . . . 5
- 6.3. Matriz de Comparación . . . 6

7. Aseguramiento de la Disponibilidad y Automatización LocalHost **7**
- 7.1. Calendario operativo . . . 7
- 7.2. Estrategia de respaldos . . . 8
  - 7.2.1. Mantenimiento preventivo automatizado . . . 11
- 7.3. Purga y rotación de datos . . . 12
- 7.4. Gestión reactiva y monitoreo . . . 14

8. Alta Disponibilidad, Replicación y Casos de Failover LocalHost **15**
- 8.1. Servidores Primario, Réplica y Balanceador usando Docker. . . . 15
  - 8.1.1. Creando el entorno . . . 15
  - 8.1.2. Visual Studio Code . . . 15
  - 8.1.3. Crea el archivo del Balanceador de Carga . . . 15
  - 8.1.4. Iniciar los servidores . . . 16
- 8.2. Configuración de Permisos para HAProxy (Health Checks) . . . 17
- 8.3. Configuración de Replicación Maestro-Esclavo (GTID) . . . 18
  - 8.3.1. Habilitar GTID y Replicación (Inyección vía Docker Compose) . . . 18
  - 8.3.2. Ejecución de la Arquitectura Limpia . . . 18
  - 8.3.3. Comandos de Replicación SQL . . . 18
  - 8.3.4. Resolución de Incidentes: Error 1396 y Conflicto GTID . . . 19
- 8.4. Cargar base de datos ECOBICI . . . 20
  - 8.4.1. Copiar los archivos CSV al contenedor Linux . . . 20
  - 8.4.2. Mover y modificar el script SQL . . . 20
  - 8.4.3. Inyectar la base de datos . . . 20

9. Crear Cuenta Google Cloud GCP. **21**
- 9.1. Crear Instancias . . . 22
  - 9.1.1. Regla de Firewall adicional . . . 24

10. Maquina Virtual Cloud GCP. **26**

11. Instalar Docker y Docker Compose Cloud GCP. **26**
- 11.1. Dar permisos a tu usuario . . . 26
  - 11.1.1. Crear la estructura del proyecto . . . 26
  - 11.1.2. Instalacion editor de texto nano . . . 26
  - 11.1.3. Crear docker-compose.yml . . . 27
  - 11.1.4. Crear haproxy.cfg . . . 27

12. Arrancar la infraestructura en la nube Cloud GCP. 27
12.1. Configuración de Permisos para HAProxy (Health Checks) . . . 27
12.1.1. Replicacion GTID . . . 28
12.1.2. Crear el usuario de clonación en el Maestro . . . 28
12.1.3. Comprobar la conexión . . . 28

13. Poblar la Base de Datos Cloud GCP. 29
13.1. Cargar Data en la Maquina Virtual GCP . . . 30
13.1.1. Copiar dentro del contenedor Maestro . . . 30
13.1.2. Ejecución del Script en MySQL Workbench . . . 31
13.1.3. PRIMARY KEY . . . 31
13.1.4. FOREIGN KEY . . . 31

14. Configuracion de Parametros Cloud GCP. 32
14.1. Configurar el nodo Maestro . . . 32
14.2. Configurar el nodo Esclavo . . . 32
14.3. Aplicar los cambios en el clúster . . . 32

15. Disponibilidad y Automatizacion Cloud GCP. 33
15.1. Instalar la herramienta cron . . . 33
15.2. Crear la carpeta y el script de respaldo . . . 33
15.2.1. Script de automatización . . . 33
15.2.2. Dar permisos y programar la tarea automática . . . 33
15.2.3. Comprobar el peso del respaldo . . . 33
15.3. Respaldo Incremental Diario (Binlogs) . . . 34
15.4. Mantenimiento Preventivo Automatizado ANALYZE y OPTIMIZE . . . 35
15.4.1. MySQL Workbench . . . 35
15.5. Rotación de Particiones y Retención de Datos . . . 36
15.6. Programación de tareas(Cron) y Monitoreo . . . 37

16. Plan de Recuperación ante Desastres (DRP) 38
16.1. Alcance y Objetivo del DRP . . . 38
16.2. Arquitectura de Respaldo Existente . . . 38
16.3. Protocolo de Respuesta (Escenarios de Desastre) . . . 39

17. Pagina web 40
17.1. Backend y Frontend . . . 40

18. Servidor Render 42

19. Plataforma de envío de correos electrónico 43

20. Maquina Viertual GCP IaaS 44
20.1. RAM configurada por nodo . . . 44

21. Consumo de RAM por contenedor 45
21.1. Distribución del consumo actual . . . 45

1. Tema y Descripción del Proyecto

1.1. Descripción General

Movilidad Urbana: Análisis del Sistema Ecobici en la Ciudad de México, consiste en el diseño, estructuración e implementación de una base de datos relacional analítica enfocada en la red de transporte de bicicletas de la CDMX. Un diseño de arquitectura basada en un Modelo de Estrella, el proyecto procesa y normaliza un volumen masivo de datos transaccionales correspondientes a los años 2024, 2025 y 2026.

1.2. Elección del Tema

Se seleccionó este tema porque el sistema Ecobici es fundamental de la movilidad sustentable y el desarrollo urbano en la capital del país. La Data proporcionada por Ecobici CDMX, genera un volumen de información masivo diariamente. Esto nos presentó un escenario ideal, para aplicar técnicas avanzadas de administración de bases de datos. Resolver las limitantes de particionamiento del motor de bases de datos mientras se garantizaba un 100 % de integridad referencial, demuestra el nivel técnico requerido para gestionar arquitecturas de datos.

1.3. Información a Visualizar en la Base de Datos

La base de datos está estructurada para visualizar la interacción entre los usuarios y el sistema de movilidad mediante las siguientes dimensiones y métricas:

- **Hechos de Movilidad:** Más de 6.5 millones de registros operativos en la tabla principal y más de 30 millones en el histórico, detallando cada viaje realizado.
- **Perfiles Demográficos:** Género y edad de los usuarios, permitiendo segmentar el comportamiento del consumidor.
- **Dimensión Espacial:** Catálogo de estaciones de retiro y arribo, incluyendo su nomenclatura y zonificación, para trazar las rutas urbanas.
- **Dimensión Temporal:** Fechas exactas y franjas horarias (Mañana, Tarde, Noche, Madrugada) para identificar horas pico y valles de demanda.
- **Inventario Físico:** Estatus y tipo de las bicicletas utilizadas.

Tabla 1: Volumetría de Datos del Proyecto

Tabla / Entidad	**Tipo en Modelo**	**Volumen Aproximado**
Historico Viajes	Tabla Particionada	$> 30,000,000$ registros
Viajes (2025-2026)	Tabla de Hechos	$> 6,500,000$ registros
Estaciones	Catálogo / Dimensión	683 estaciones únicas
Tiempos	Dimensión Temporal	86,400 segundos

1.4. Finalidad Principal

La finalidad principal de esta base de datos no es únicamente el almacenamiento, es servir como un repositorio analítico. El objetivo es transformar los datos en información estratégica que permita a la toma de decisiones optimizar la logística del sistema. Mediante las sentencias DML, la base de datos es capaz de responder preguntas de negocio en segundos. Ejemplo: la saturación de las estaciones por franjas horarias, los patrones de ruta más frecuentes según el género del usuario.

2. Diagrama Entidad-Relación

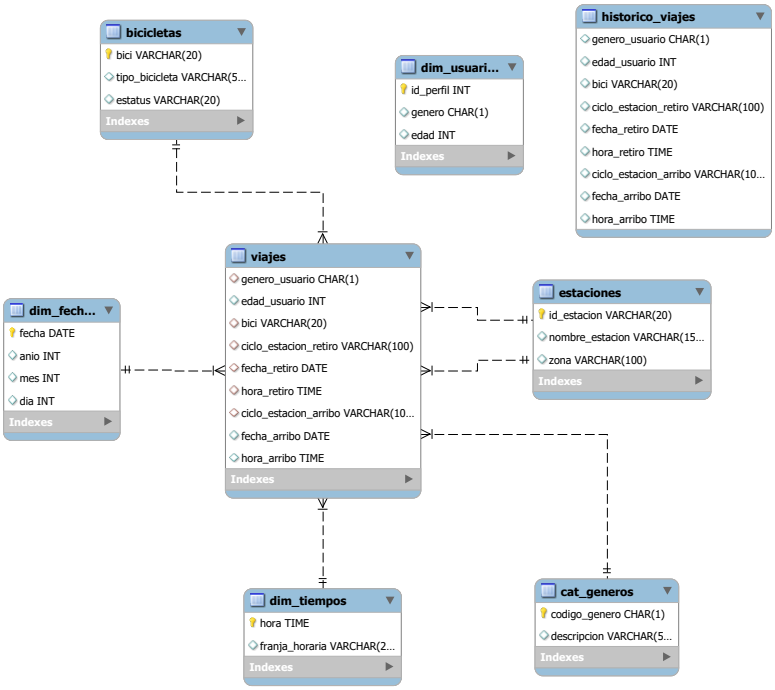


Figura 1: Diagrama Entidad-Relación (Modelo de Estrella) estructurado para el análisis.

3. Enlace a Repositorio GitHub.

Repositorio GitHub

4. Enlace Google Drive

DATA DRIVE

5. Estimación Teórica y Proyección de Espacio en Disco LocalHost.

5.1. 1. Tamaño por tipo de dato y fila (Row Size)

Para realizar la estimación de espacio, se sumaron los bytes requeridos por cada columna de las tablas principales (`viajes`, `historico_viajes`, `dim_usuarios`, `estaciones`, etc.), utilizando los pesos estándar de MySQL: `INT` = 4 bytes, `DATETIME` = 5 bytes, `SMALLINT` = 2 bytes y `TINYINT` = 1 byte.

A esto se sumó la sobrecarga (*overhead*) interna de InnoDB: encabezado de fila (∼5 bytes), ID de transacción (6 bytes) y puntero de rollback (7 bytes). Considerando el tamaño por defecto de la página InnoDB (16 KB) y un *fill factor* de 15/16, se determinó que la fila promedio ocupa aproximadamente **70 bytes** (incluyendo índices B-Tree de las Claves Primarias y Foráneas).

5.2. 2. Cálculo del Espacio de Carga Inicial

Con la fila promedio de 70 bytes, se proyectó la carga inicial antes de la inyección física de datos:

- **Carga Inicial de `historico_viajes`** (∼30,000,000 registros): ∼2.1 GB de datos + ∼1.2 GB de índices = ∼**3.3 GB**.
- **Carga Inicial de `viajes`** (∼6,500,000 registros): ∼455 MB de datos + ∼260 MB de índices = ∼**715 MB**.
- **Catálogos y Dimensiones:** ∼**15 MB**.
- **Total Carga Inicial:** ∼**4.03 GB**.

5.3. 3. Simulación y Proyección de Crecimiento

Asumiendo un promedio de 800,000 viajes mensuales nuevos en la CDMX y un incremento anual estimado del 10 % debido a nuevas estaciones. El crecimiento mensual proyectado es: 800,000 × 100 bytes (datos + índices) ≈ 80 MB/mes (lo que equivale a ∼0.96 GB/año).

Periodo	**Almacenamiento Estimado**
Año 1 (Actual / Carga Inicial)	4.03 GB
Año 2 (+10 % uso)	∼5.08 GB
Año 3 (+20 % uso)	∼6.24 GB
Año 5 (+40 % uso)	∼8.95 GB

Tabla 2: Tabla de Proyección a 5 Años

6. Consulta Diccionario de Datos y Comparación de Almacenamiento LocalHost.

6.1. Justificación Teórica

En MySQL 8.0, los metadatos se almacenan de forma centralizada y transaccional en el Diccionario de Datos. Medir los archivos físicos (como `.ibd`) directamente desde el sistema operativo resulta impreciso debido a dos factores:

- **Asignación por Extents:** El motor InnoDB reserva espacio en bloques completos, por lo que el archivo en disco siempre será más grande que los datos reales almacenados.
- **Fragmentación:** El espacio de los registros eliminados no se devuelve inmediatamente al sistema operativo.

Por ello, consultar el `INFORMATION_SCHEMA` permite al administrador medir el espacio lógico real, exacto y en uso.

6.2. Desarrollo de Consultas SQL

A. Tamaño total de la Base de Datos

```
    SELECT table_schema AS Base_de_Datos,
    ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS Tamano_MB
    FROM information_schema.tables
    WHERE table_schema = 'DB_Ecobici_CDMX'
    GROUP BY table_schema;
```

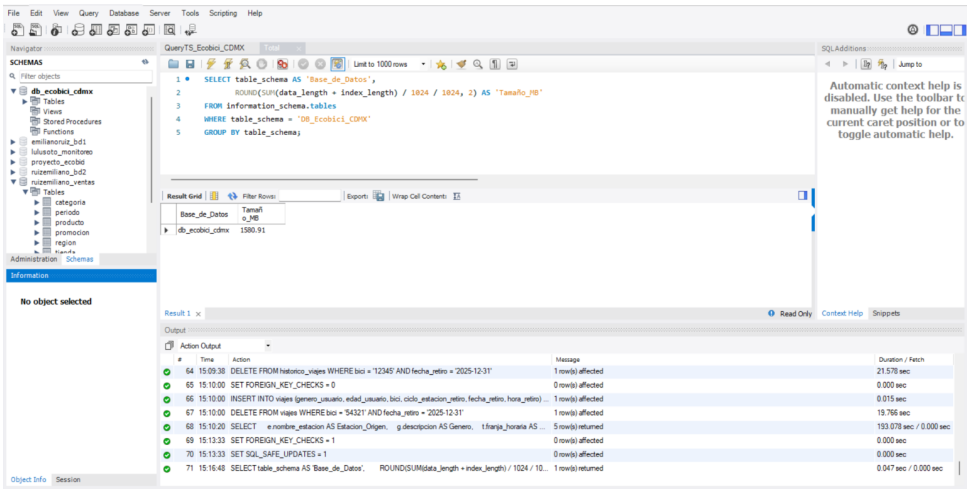


Figura 2: Tamaño total de la base de datos.

Como se observa en el resultado, el tamaño lógico total del esquema `DB_Ecobici_CDMX` es de **1580.91 MB** (aproximadamente 1.58 GB). Este valor representa la suma exacta de los datos puros más el peso de los índices, confirmando que la carga masiva de los registros se completó exitosamente.

B. Tamaño individual por tabla e índice

```
SELECT table_name AS `Tabla`,
       ROUND(data_length / 1024 / 1024, 2) AS `Datos_MB`,
       ROUND(index_length / 1024 / 1024, 2) AS `Indices_MB`,
       table_rows AS `Numero_Registros`
FROM information_schema.tables
WHERE table_schema = 'DB_Ecobici_CDMX'
ORDER BY (data_length + index_length) DESC;
```

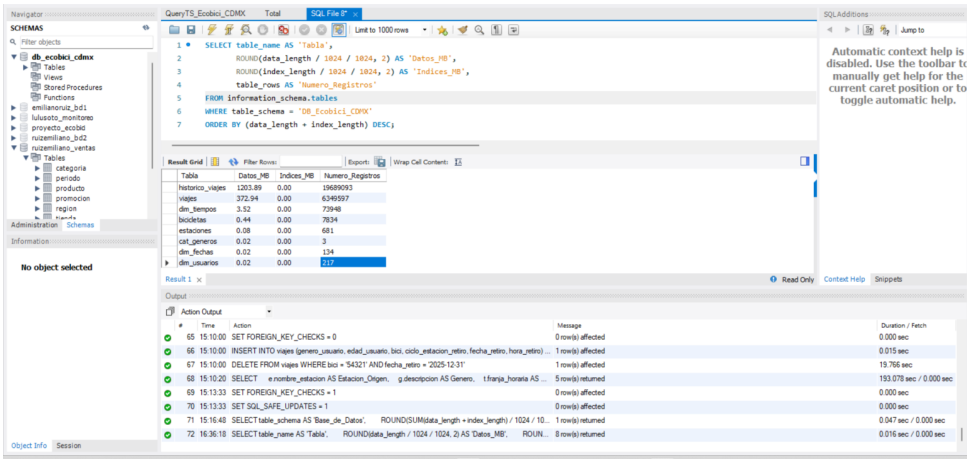


Figura 3: Desglose de peso por datos e índices de cada tabla.

C. Consulta de Tablespaces y archivos .ibd

```sql
SELECT TABLESPACE_NAME AS `Particion_Tablespace`,
       FILE_NAME AS `Ruta_Archivo_ibd`,
       ROUND(TOTAL_EXTENTS * EXTENT_SIZE / 1024 / 1024, 2) AS `Espacio_Asignado_MB`,
       ROUND(FREE_EXTENTS * EXTENT_SIZE / 1024 / 1024, 2) AS `Espacio_Libre_MB`
FROM information_schema.FILES
WHERE TABLESPACE_NAME LIKE 'DB_Ecobici_CDMX/historico_viajes%';
```

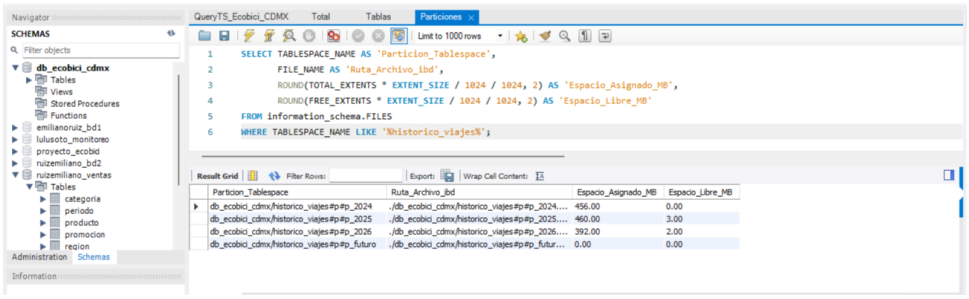


Figura 4: Espacio físico asignado vs libre en las particiones de la tabla histórica.

6.3. Matriz de Comparación

El siguiente cuadro contrasta la estimación matemática contra la medición real del Diccionario de Datos.

Tabla	Filas (Aprox)	Teórico (MB)	Real (MB)	Diferencia
historico_viajes	19,689,093	3379.20	1203.89	-2175.31
viajes	6,349,597	715.00	372.94	-342.06
Catálogos (Suma)	82,683	15.00	4.10	-10.90

Tabla 3: Comparativa Estimación Teórica vs Diccionario de Datos

Análisis: El tamaño real resultó menor a la estimación teórica principalmente porque el modelo matemático calculó la tabla histórica sobre 30 millones de registros, mientras que la carga física procesó aproximadamente 20 millones.

7. Aseguramiento de la Disponibilidad y Automatización LocalHost

La replicación configurada en la Sección 1.3 protege al sistema contra la caída de un servidor, pero no contra errores lógicos. Por ejemplo, si un usuario ejecuta por error un `DELETE` sobre la tabla de viajes, la réplica aplica ese mismo borrado en cuestión de segundos. Por esta razón, la replicación no sustituye a los respaldos.

Para garantizar la continuidad y el mantenimiento del motor, se diseñó una estrategia operativa automatizada con cuatro ejes: respaldos programados, mantenimiento preventivo, purga y rotación de datos, y monitoreo. Las tareas se programan durante la madrugada, cuando la operación de Ecobici es mínima, y se ordenan para evitar que varios procesos pesados coincidan.

7.1. Calendario operativo

Tarea	**Herramienta**	**Frecuencia**	**Hora (CDMX)**
Respaldo incremental (binlogs)	`cron` + `docker cp`	Diario	00:50
Respaldo completo	`cron` + `mysqldump`	Semanal (domingo)	03:00
Rotación de particiones	`cron`	Mensual (primer sábado)	01:00
`ANALYZE TABLE`	Event Scheduler	Semanal (sábado)	03:00
`OPTIMIZE TABLE viajes`	Event Scheduler	Mensual (primer sábado)	03:00
Purga de binlogs	MySQL (automática)	Continua, 14 días	—
Reporte de monitoreo	`cron`	Diario	07:00

Tabla 4: Calendario de tareas automatizadas.

El primer sábado de cada mes, la rotación de particiones se ejecuta a la 01:00 y el mantenimiento a las 03:00. Este orden permite que el mantenimiento se ejecute después de la rotación. El respaldo completo del domingo se programa en un día distinto al mantenimiento del sábado.

7.2. Estrategia de respaldos

La estrategia combina dos tipos de respaldo: un respaldo lógico **completo** cada semana y una copia diaria de los registros binarios (*binary logs* o *binlogs*). El respaldo completo representa una fotografía de la base de datos; los binlogs registran los cambios posteriores. En conjunto, y siempre que la cadena de archivos esté completa y sea consistente, permiten recuperar la información hasta un punto determinado en el tiempo (*Point-in-Time Recovery*, PITR).

Respaldo completo semanal. El respaldo se ejecuta con `mysqldump` desde la réplica para reducir la carga de lectura sobre el servidor principal. Las opciones exactas deben validarse con la versión de MySQL utilizada y con la configuración GTID del entorno. El ejemplo siguiente presupone que las credenciales se gestionan de forma segura y que los directorios de destino ya existen.

```
#!/bin/bash
# Respaldo logico completo semanal (domingo 03:00)
set -o pipefail

FECHA=$(date +%Y-%m-%d)
DIR="$HOME/proyecto_ecobici"
ARCHIVO="$DIR/backups/completos/ecobici_completo_${FECHA}.sql.gz"

if docker exec mysql-slave mysqldump \
    --defaults-extra-file=/run/secrets/mysql-client.cnf \
    --single-transaction --source-data=2 --set-gtid-purged=ON \
    --routines --triggers --events \
    --databases DB_Ecobici_CDMX | gzip > "$ARCHIVO"; then
    echo "$(date) OK respaldo completo: $ARCHIVO" >> "$DIR/logs/respaldos.log"
else
    echo "$(date) ERROR en respaldo completo" >> "$DIR/logs/respaldos.log"
    rm -f "$ARCHIVO"
    exit 1
fi

# Conservar aproximadamente las ultimas cuatro semanas
find "$DIR/backups/completos" \
    -name "ecobici_completo_*.sql.gz" -mtime +28 -delete
```

Nota de seguridad: el archivo de credenciales del ejemplo es ilustrativo. Debe existir dentro del contenedor, tener permisos restrictivos y configurarse según la imagen utilizada. No se recomienda guardar contraseñas directamente en el código ni publicarlas en el repositorio.

Respaldo diario de binlogs. Cada noche se puede ejecutar `FLUSH BINARY LOGS` para cerrar el binlog actual y comenzar uno nuevo. Los archivos cerrados se copian fuera del contenedor; el archivo activo no debe copiarse mientras MySQL siga escribiéndolo. El ejemplo requiere que el usuario de MySQL tenga los permisos necesarios y que el directorio de destino exista.

```
#!/bin/bash
# Copia diaria de binlogs (00:50)
set -euo pipefail

DIR="$HOME/proyecto_ecobici"
DESTINO="$DIR/backups/binlogs"
mkdir -p "$DESTINO"

# Se recomienda usar un archivo de opciones protegido.
MYSQL=(docker exec mysql-master mysql
       --defaults-extra-file=/run/secrets/mysql-client.cnf
       -N -e)

# 1. Rotar el binlog activo
"${MYSQL[@]}" "FLUSH BINARY LOGS;"

# 2. Identificar el archivo binlog activo
ACTIVO=$("${MYSQL[@]}" "SHOW BINARY LOG STATUS;" | awk 'NR==1 {print $1}')

# 3. Copiar binlogs cerrados que aun no existan en el destino
for f in $(docker exec mysql-master \
    sh -c "ls /var/lib/mysql | grep -E '^mysql-bin\\.[0-9]+$'"); do
    if [ "$f" != "$ACTIVO" ] && [ ! -f "$DESTINO/$f" ]; then
        docker cp "mysql-master:/var/lib/mysql/$f" "$DESTINO/"
        echo "$(date) Binlog copiado: $f" >> "$DIR/logs/respaldos.log"
    fi
done

# 4. Aplicar la retencion definida para las copias locales
find "$DESTINO" -type f -name "mysql-bin.*" -mtime +14 -delete
```

Importante: el nombre de la consulta de estado depende de la versión de MySQL. En versiones que no admitan `SHOW BINARY LOG STATUS`, debe utilizarse la sentencia equivalente disponible en esa versión. Además, la rotación y copia de binlogs debe coordinarse con la política de purga del servidor y con el estado de la réplica.

Procedimiento de recuperación a un punto en el tiempo (PITR). Supóngase que el miércoles 21 de octubre de 2026 a las 15:00, hora de Ciudad de México, se ejecuta por error un `DELETE` que elimina los viajes de octubre. La recuperación debe realizarse en una instancia aislada, sin modificar directamente el servidor principal.

Listing 1: Esquema de recuperación a un punto en el tiempo

```
# 1. Preparar una instancia aislada con la misma version compatible
# de MySQL y una configuracion de recuperacion validada.
docker run -d --name mysql-recuperacion \
    -e MYSQL_ROOT_PASSWORD='CAMBIAR_POR_UN_SECRETO_SEGURO' \
    mysql:8.0

# 2. Restaurar el respaldo completo correspondiente.
# El procedimiento exacto depende de las opciones GTID incluidas
# en el dump y del estado inicial de la instancia de recuperacion.
gunzip -c backups/completos/ecobici_completo_2026-10-18.sql.gz \
    | docker exec -i mysql-recuperacion \
      mysql -uroot -p

# 3. Copiar los binlogs disponibles al contenedor de recuperacion.
docker cp backups/binlogs/. mysql-recuperacion:/tmp/binlogs/

# 4. Inspeccionar y reproducir los binlogs en orden cronologico,
# deteniendose justo antes del momento del error.
# Verificar primero la zona horaria y el formato temporal de los logs.
# Ejemplo ilustrativo: ajustar los nombres de archivo y la hora real.
docker exec mysql-recuperacion sh -c \
  "mysqlbinlog --stop-datetime='2026-10-21 20:59:59' \
  /tmp/binlogs/mysql-bin.000001 /tmp/binlogs/mysql-bin.000002 \
  | mysql -uroot -p"
```

El comando anterior es un esquema de procedimiento, no una receta universal que deba ejecutarse sin validación. Antes de reproducir los binlogs, se debe confirmar su orden, la continuidad de la secuencia, la zona horaria y la compatibilidad de los GTID incluidos en el respaldo. La restauración de un dump que contiene `SET @@GLOBAL.GTID_PURGED` puede requerir una instancia preparada con el historial GTID apropiado; no se debe ejecutar `RESET MASTER` de forma automática sin revisar las consecuencias para la recuperación.

Una vez validada la información restaurada, los registros recuperados se exportan de la instancia aislada y se reintegran al servidor principal mediante un procedimiento controlado, con respaldo previo y validación de claves e integridad.

Retención de respaldos. La política propuesta conserva cuatro semanas de respaldos completos y 14 días de copias de binlogs. Esta retención solo es suficiente si los respaldos completos y los binlogs necesarios para llegar al punto de recuperación deseado están disponibles y verificados. Los archivos de respaldo deben almacenarse, preferentemente, en un destino independiente del servidor que se protege. La duración de retención configurada en MySQL debe coordinarse con las copias externas; la purga del servidor no debe eliminar archivos que aún sean necesarios para la réplica o para la recuperación.

7.2.1. Mantenimiento preventivo automatizado

Las operaciones de inserción, actualización y borrado pueden provocar cambios en la distribución de las páginas y en el uso del espacio de almacenamiento. Por ello, el mantenimiento debe basarse en métricas y en las características de la tabla, no únicamente en la suposición de que toda tabla necesita reconstruirse periódicamente.

- `ANALYZE TABLE` actualiza estadísticas que el optimizador utiliza para seleccionar planes de ejecución. Puede programarse periódicamente y comprobarse su impacto en la carga.
- `OPTIMIZE TABLE` puede reconstruir una tabla InnoDB y compactar su almacenamiento; su duración, espacio temporal y bloqueo dependen de la versión, el tamaño de la tabla y la operación concreta. Por ello, debe ejecutarse en una ventana de mantenimiento y después de evaluar su necesidad.

La tabla `viajes` recibe operaciones transaccionales y puede experimentar cambios frecuentes. Si no tiene una llave primaria explícita, InnoDB crea un índice agrupado interno, pero se recomienda definir una llave primaria adecuada para facilitar la identificación de filas, mejorar el diseño y reducir problemas de mantenimiento y replicación. En cambio, si `historico_viajes` recibe datos por lotes y se mantiene principalmente como histórico, el particionamiento puede facilitar la eliminación de periodos completos. La eliminación de una partición debe realizarse únicamente después de resumir, exportar y verificar los datos que contiene.

En MySQL 8.0, el Event Scheduler puede estar habilitado o deshabilitado según la configuración del servidor. Su estado se verifica con:

Listing 2: Verificación del Event Scheduler

```
SHOW VARIABLES LIKE 'event_scheduler';
```

El siguiente ejemplo ilustra un evento semanal. La fecha de inicio debe ajustarse al calendario real y a la zona horaria configurada en el servidor. Para que el mantenimiento mensual ocurra el primer sábado, es preferible programar un evento mensual separado, en lugar de depender de una condición dentro de un evento semanal.

Listing 3: Evento de mantenimiento semanal

```
USE DB_Ecobici_CDMX;

CREATE EVENT IF NOT EXISTS ev_analisis_semanal
ON SCHEDULE EVERY 1 WEEK
STARTS '2026-10-31 09:00:00'
COMMENT 'Actualizacion semanal de estadisticas'
DO
    ANALYZE TABLE viajes, historico_viajes;
```

Listing 4: Evento mensual de optimización

```
USE DB_Ecobici_CDMX;

CREATE EVENT IF NOT EXISTS ev_optimizacion_mensual
ON SCHEDULE EVERY 1 MONTH
STARTS '2026-11-07 09:00:00'
COMMENT 'Evaluar y optimizar la tabla viajes mensualmente'
DO
    OPTIMIZE TABLE viajes;
```

Los horarios anteriores son ejemplos y deben verificarse con la zona horaria efectiva de MySQL. Además, se debe confirmar que `event_scheduler` esté en `ON` y que el usuario que crea los eventos tenga el privilegio `EVENT`. Los eventos creados en el servidor principal se replican en la réplica con estado deshabilitado para evitar su ejecución duplicada; por ello, no debe asumirse que las sentencias de mantenimiento se ejecutarán automáticamente en ambos nodos.

Para verificar el estado de los eventos y revisar el espacio libre reportado por el diccionario de datos, se pueden ejecutar las siguientes consultas:

Listing 5: Verificación de eventos y espacio libre

```
SELECT EVENT_NAME, STATUS, LAST_EXECUTED
FROM information_schema.EVENTS
```

```sql
WHERE EVENT_SCHEMA = 'DB_Ecobici_CDMX';

SELECT TABLE_NAME AS Tabla,
       ROUND(DATA_FREE / 1024 / 1024, 2) AS Espacio_Libre_MB
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'DB_Ecobici_CDMX'
ORDER BY DATA_FREE DESC;
```

El valor de `DATA_FREE` es una estimación del espacio libre asignado o disponible según el motor y el almacenamiento; no constituye por sí solo una medición concluyente de fragmentación. Conviene acompañarlo con métricas de tamaño, rendimiento y duración de consultas antes y después del mantenimiento.

7.3. Purga y rotación de datos

Purga de registros binarios. La retención automática de binlogs se configura mediante la variable correspondiente de MySQL. Para un periodo de 14 días, el valor en segundos es:

$$14 \times 24 \times 60 \times 60 = 1\,209\,600.$$

En las versiones que utilicen esta variable, la configuración puede expresarse como `binlog_expire_logs_seconds=1209`
La purga manual equivalente se puede solicitar con:

```sql
PURGE BINARY LOGS BEFORE NOW() - INTERVAL 14 DAY;
```

Antes de una purga manual se debe comprobar que la réplica ya no necesite los archivos que se eliminarán. Si una réplica permanece desconectada y se purgan los binlogs que aún necesita, puede perder la capacidad de continuar la replicación y requerir una resincronización.

Política de retención de viajes. Los viajes detallados pueden ser relevantes durante los primeros meses; después, suelen consultarse principalmente totales y tendencias, como la demanda por estación, la estacionalidad y el perfil general de los usuarios. Para conciliar ambas necesidades, se proponen dos técnicas: **resumir** la información esencial dentro de la base de datos y **archivar** el detalle completo fuera de ella.

Información	**Ubicación**	**Retención**
Viajes detallados	`historico_viajes` (base activa)	3 meses
Resumen mensual por estación y perfil	`resumen_viajes_mes`	Permanente, sujeto a política de conservación
Detalle histórico	Google Cloud Archive Storage	Archivo de largo plazo
Respaldos completos	Servidor (`backups/completos`)	4 semanas
Binlogs	Servidor y `backups/binlogs`	14 días

Tabla 5: Política propuesta de retención de datos.

El resumen permanece en MySQL porque Archive Storage es almacenamiento de objetos, no una base de datos relacional: los objetos no se consultan directamente mediante SQL y las recuperaciones pueden tener costes y latencia adicionales. Además, una tabla agregada puede ocupar mucho menos espacio que el detalle de todos los viajes.

Alcance del resumen. Si los datos abiertos de Ecobici no contienen un identificador individual de usuario ni la distancia recorrida, y la tabla `estaciones` no contiene coordenadas, el resumen debe limitarse a las variables realmente disponibles. En este diseño se propone agrupar por estación de retiro, género y mes, e incluir la edad promedio y la duración promedio de los viajes. Un resumen anual personalizado para cada usuario requeriría un identificador de usuario consistente y autorizado.

Listing 6: Tabla de resumen mensual

```sql
CREATE TABLE resumen_viajes_mes (
    anio                SMALLINT NOT NULL,
    mes                 TINYINT NOT NULL,
    ciclo_estacion_retiro VARCHAR(100) NOT NULL,
    genero_usuario      CHAR(1) NOT NULL,
    total_viajes        INT NOT NULL,
    edad_promedio       DECIMAL(4,1),
    duracion_promedio_min DECIMAL(6,1),
```

```
    PRIMARY KEY (anio, mes, ciclo_estacion_retiro, genero_usuario)
);
```

En el código de producción, el valor que represente datos nulos de género debe documentarse y normalizarse en el proceso de carga. También conviene validar que la clave de agrupación represente de forma única a la estación y no dependa únicamente de un nombre que pueda cambiar.

Transición a particiones mensuales. Si `historico_viajes` está particionada por año mediante `RANGE COLUMNS(fecha_retiro)`, una partición anual no permite aplicar directamente una retención de tres meses: `DROP PARTITION` eliminaría el año completo. Por ello, se propone reorganizar la partición del año en curso en particiones mensuales y crear anticipadamente las particiones futuras. Antes de aplicar el cambio, se debe verificar la definición real de la tabla, el tipo de la columna de fecha y las restricciones de particionamiento de la versión instalada.

Listing 7: División de la partición de 2026 en meses

```
ALTER TABLE historico_viajes
REORGANIZE PARTITION p_2026 INTO (
    PARTITION p_202601 VALUES LESS THAN ('2026-02-01'),
    PARTITION p_202602 VALUES LESS THAN ('2026-03-01'),
    PARTITION p_202603 VALUES LESS THAN ('2026-04-01'),
    PARTITION p_202604 VALUES LESS THAN ('2026-05-01'),
    PARTITION p_202605 VALUES LESS THAN ('2026-06-01'),
    PARTITION p_202606 VALUES LESS THAN ('2026-07-01'),
    PARTITION p_202607 VALUES LESS THAN ('2026-08-01'),
    PARTITION p_202608 VALUES LESS THAN ('2026-09-01'),
    PARTITION p_202609 VALUES LESS THAN ('2026-10-01'),
    PARTITION p_202610 VALUES LESS THAN ('2026-11-01'),
    PARTITION p_202611 VALUES LESS THAN ('2026-12-01'),
    PARTITION p_202612 VALUES LESS THAN ('2027-01-01')
);
```

El resultado se comprueba mediante el diccionario de datos:

Listing 8: Verificación de particiones

```
SELECT PARTITION_NAME, PARTITION_DESCRIPTION, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_SCHEMA = 'DB_Ecobici_CDMX'
  AND TABLE_NAME = 'historico_viajes'
ORDER BY PARTITION_ORDINAL_POSITION;
```

Las particiones anuales `p_2024` y `p_2025` estarían fuera del periodo activo si la política de retención comienza en 2026. Su archivo y eliminación deben ejecutarse únicamente después de exportar, verificar y respaldar sus datos. Si existe un script de rotación, se debe comprobar que sus parámetros, consultas y operaciones correspondan a la definición real de las particiones.

Listing 9: Ejemplo de ejecución manual de rotación

```
./rotacion_particiones.sh p_2024
./rotacion_particiones.sh p_2025
```

Proceso mensual de rotación. La regla central del proceso es que **la operación irreversible va al final**. `DROP PARTITION` elimina los datos de la partición; por ello, solo debe ejecutarse después de resumir, exportar y verificar la información. Si cualquier paso falla, el proceso debe detenerse sin borrar datos y registrar el error en la bitácora.

1. Calcular el resumen mensual e insertarlo en `resumen_viajes_mes`.
2. Exportar el detalle de la partición a un archivo CSV.
3. Verificar que el archivo contenga el mismo número de registros que la partición.
4. Comprimir el archivo, subirlo a Archive Storage y comprobar que la carga terminó correctamente.
5. Eliminar la partición con `DROP PARTITION`, después de validar el archivo y el resumen.
6. Crear la partición del mes siguiente si aún no existe.

Si la partición está vacía, los pasos de exportación pueden omitirse después de comprobar explícitamente que no contiene registros. El bucket de Archive Storage debe crearse una sola vez, en la región elegida para el proyecto, y debe configurarse con permisos y controles de acceso apropiados.

7.4. Gestión reactiva y monitoreo

Error Log. En los contenedores, los mensajes de MySQL pueden consultarse mediante `docker logs`. La disponibilidad de la tabla `performance_schema.error_log` depende de la versión y la configuración de MySQL. Si está disponible, puede utilizarse una consulta como la siguiente:

Listing 10: Consulta ilustrativa de errores y advertencias

```
SELECT LOGGED, PRIO, ERROR_CODE, LEFT(DATA, 120) AS mensaje
FROM performance_schema.error_log
WHERE PRIO IN ('Error', 'Warning')
  AND LOGGED > NOW() - INTERVAL 1 DAY
ORDER BY LOGGED DESC;
```

Slow Query Log. Si el registro de consultas lentas está habilitado y configurado para almacenar sus resultados en tablas, se pueden revisar las consultas que tardan más. El umbral de dos segundos debe establecerse en la configuración del servidor; no se activa solo por consultar la tabla de registro.

Listing 11: Consulta ilustrativa de las consultas lentas

```
SELECT start_time, query_time, rows_examined,
       LEFT(sql_text, 100) AS consulta
FROM mysql.slow_log
ORDER BY query_time DESC
LIMIT 10;
```

Memoria del Buffer Pool. El parámetro `innodb_buffer_pool_size` define la memoria que InnoDB utiliza para almacenar en caché páginas de datos e índices. Su valor adecuado depende de la RAM disponible, la carga de trabajo y los límites de memoria de los contenedores. En este entorno se propone asignar 1 GiB por nodo, siempre que la suma de límites y el consumo real de los procesos que comparten la máquina lo permitan.

Listing 12: Tamaño y métricas del Buffer Pool

```
SELECT @@innodb_buffer_pool_size / 1024 / 1024 AS Buffer_Pool_MB;

SELECT ROUND(
    100 * (1 - r.VARIABLE_VALUE / q.VARIABLE_VALUE), 2
) AS Hit_Ratio_Pct
FROM performance_schema.global_status AS r
JOIN performance_schema.global_status AS q
  ON q.VARIABLE_NAME = 'Innodb_buffer_pool_read_requests'
WHERE r.VARIABLE_NAME = 'Innodb_buffer_pool_reads';

SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM performance_schema.global_status
WHERE VARIABLE_NAME IN (
    'Innodb_buffer_pool_pages_free',
    'Innodb_buffer_pool_pages_total',
    'Created_tmp_disk_tables',
    'Sort_merge_passes',
    'Select_full_join'
);
```

El cociente de aciertos del Buffer Pool debe interpretarse como una señal orientativa y evaluarse junto con la carga de trabajo; no existe un umbral universal que, por sí solo, demuestre un problema.

Observación sobre la replicación. Las tablas del proyecto deben contar con claves que permitan identificar las filas de forma eficiente. Cuando se usa replicación basada en filas (`ROW`), la ausencia de una llave primaria o de un índice adecuado puede incrementar el coste de localizar las filas afectadas por ciertas operaciones. Esto puede contribuir al retraso de la réplica, que debe vigilarse junto con el estado de los hilos de replicación, los errores y el volumen de transacciones pendientes. Se recomienda revisar las claves primarias propuestas en el script del proyecto y validar su impacto antes de incorporarlas al entorno activo.

Indicador	Señal que conviene investigar
Hit Ratio	Una tasa persistentemente baja puede indicar lecturas frecuentes desde disco o un conjunto de trabajo mayor que la caché.
`Innodb_buffer_pool_pages_free`	Un valor cercano a cero durante periodos prolongados requiere revisar la presión de memoria y la actividad de InnoDB.
`Created_tmp_disk_tables`	Un crecimiento elevado puede indicar consultas que generan tablas temporales en disco.
`Sort_merge_passes`	Un incremento sostenido puede indicar ordenamientos que requieren varias pasadas.
`Select_full_join`	Un valor creciente puede revelar uniones sin índices adecuados.

Tabla 6: Interpretación orientativa de indicadores de memoria y consultas.

8. Alta Disponibilidad, Replicación y Casos de Failover LocalHost

8.1. Servidores Primario, Réplica y Balanceador usando Docker.

8.1.1. Creando el entorno

Terminal de WSL (Ubuntu). Crear la estructura de carpetas exacta que necesitamos.

Listing 13: Creación de la estructura de directorios

```
mkdir -p ~/proyecto_ecobici/haproxy \
         ~/proyecto_ecobici/master/conf/mysql.conf.d \
         ~/proyecto_ecobici/slave/conf/mysql.conf.d
```

```
cd ~/proyecto_ecobici
```

8.1.2. Visual Studio Code

En un nuevo archivo en la raíz de la carpeta llámado docker-compose.yml.

8.1.3. Crea el archivo del Balanceador de Carga

En VS Code, en la carpeta haproxy, se crea un archivo llamado haproxy.cfg.
Terminal de terminal de WSL en caso se problemas por caracteres

```
    cat << 'EOF' > haproxy/haproxy.cfg
global
    log stdout format raw local0
    maxconn 1000

defaults
    log     global
    mode    tcp
    option  tcplog
    timeout connect 5s
    timeout client  300s
    timeout server  300s

frontend mysql_frontend
    bind *:3306
    default_backend mysql_backend

backend mysql_backend
    mode tcp
    option mysql-check user root
    server master mysql-master:3306 check inter 2000 rise 2 fall 3
    server slave mysql-slave:3306 check backup inter 2000 rise 2 fall 3

listen stats
    bind *:8404
    mode http
    stats enable
```

```
    stats uri /stats
    stats refresh 5s
EOF
```

8.1.4. Iniciar los servidores

```
    docker compose up -d
```

```
    http://localhost:8404/stats.
```

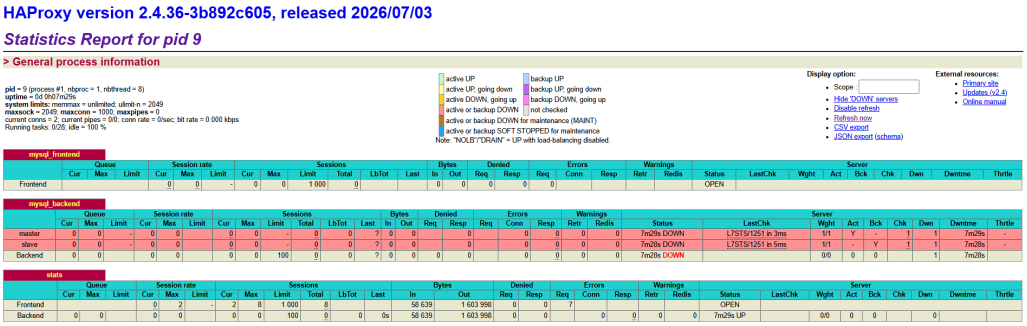


Figura 5: Localhost

8.2. Configuración de Permisos para HAProxy (Health Checks)

Se creó un usuario dedicado sin contraseña y sin privilegios, cuyo único propósito es responder al ping del balanceador.

1. Actualizar la configuración de HAProxy En el archivo `haproxy.cfg`, se sustituyó el usuario `root`:

```
    option mysql-check user root

    option mysql-check user haproxy_check
```

2. Crear el usuario en los motores MySQL Debido a incompatibilidades de encriptación entre HAProxy y MySQL 8.0, se forzó el método de autenticación `mysql_native_password` al crear el usuario en ambos contenedores.

Para el Maestro:

```
docker exec -it mysql-master mysql -uroot -p-e "CREATE USER 'haproxy_check'@'%'
    IDENTIFIED WITH mysql_native_password BY ''; FLUSH PRIVILEGES;"
```

Para la Réplica:

```
docker exec -it mysql-slave mysql -uroot -p -e "CREATE USER 'haproxy_check'@'%'
    IDENTIFIED WITH mysql_native_password BY ''; FLUSH PRIVILEGES;"
```

3. Aplicar los cambios en el Balanceador Se reinició el servicio para que HAProxy adoptara el nuevo usuario:

```
docker compose restart haproxy
```

Tras la ejecución, el panel de estadísticas en `http://localhost:8404/stats` confirmó que ambos nodos pasaron a estado activo (UP).

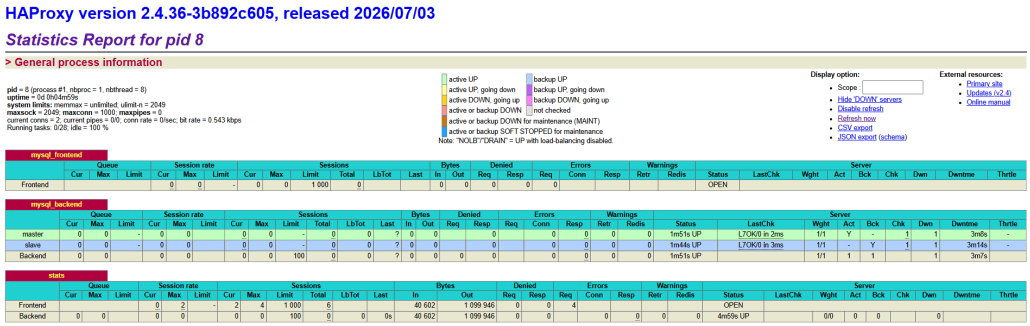


Figura 6: Localhost acceso a la base de datos (Estado UP)

8.3. Configuración de Replicación Maestro-Esclavo (GTID)

Para el cumplimiento del diseño de Alta Disponibilidad, se implementó replicación asíncrona mediante GTID (Global Transaction Identifier). GTID es el estándar moderno en MySQL que facilita la sincronización, ya que cada transacción recibe un ID único y la réplica sabe exactamente qué eventos procesar sin requerir posiciones exactas de archivos `binlog`.

8.3.1. Habilitar GTID y Replicación (Inyección vía Docker Compose)

Debido a restricciones de seguridad en los permisos de volúmenes compartidos entre Windows y WSL (donde el motor ignora archivos de configuración con permisos demasiado abiertos), se optó por prescindir de los archivos físicos `mysqld.cnf`.

En su lugar, las variables de entorno para habilitar GTID se inyectaron directamente en la definición de cada contenedor dentro del archivo `docker-compose.yml` utilizando la directiva `command`.

Configuración del Maestro en docker-compose.yml

```
    command: --server-id=1 --log-bin=mysql-bin --binlog-format=ROW --gtid-mode=ON --
    enforce-gtid-consistency=ON
```

Configuración de la Réplica en docker-compose.yml

```
    command: --server-id=2 --gtid-mode=ON --enforce-gtid-consistency=ON --read-only=1
```

8.3.2. Ejecución de la Arquitectura Limpia

Para purgar configuraciones residuales y metadatos corruptos, se procedió a destruir el entorno anterior (incluyendo los volúmenes físicos de datos con el flag `-v`) y a instanciar la infraestructura en limpio:

```
docker compose down -v
docker compose up -d
```

8.3.3. Comandos de Replicación SQL

1. Obtener la IP interna del servidor Maestro La Réplica requiere enlazarse directamente a la dirección IP privada del contenedor Maestro dentro de la red virtual de Docker (`ecobici-net`).

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mysql-master
```

Resultado: `172.18.xxx`

2. Crear el usuario de replicación en el Maestro

```
docker exec -it mysql-master mysql -uroot -p_xxxx
```

```
CREATE USER 'replica_user'@'%' IDENTIFIED WITH mysql_native_password BY 'xxx';
GRANT REPLICATION SLAVE ON *.* TO 'replica_user'@'%';
FLUSH PRIVILEGES;
EXIT;
```

3. Conectar la Réplica al Maestro

```
docker exec -it mysql-slave mysql -uroot -p_xxx
```

```
CHANGE MASTER TO
  MASTER_HOST='xxx',
  MASTER_USER='replica_user',
  MASTER_PASSWORD='xxx',
  MASTER_AUTO_POSITION=1;
START SLAVE;
```

8.3.4. Resolución de Incidentes: Error 1396 y Conflicto GTID

Durante el proceso de conexión (`START SLAVE;`), el *Coordinator worker* detuvo la clonación arrojando el error `Last_SQL_Errno: 1396` (`Operation CREATE USER failed`). Esto ocurrió porque el servidor esclavo intentó replicar una transacción DDL de creación de usuario que ya existía físicamente en su propio disco.

Para desatascar el flujo sin reconstruir el entorno, se aplicó una técnica avanzada de DBA inyectando un GTID vacío correspondiente a la transacción conflictiva, instruyendo al motor a omitirla y continuar con la cola de eventos.

```
STOP SLAVE;
SET GTID_NEXT="a8d9d588-cea3-11f1-ad19-0a3abf4f4181:7";
BEGIN;
COMMIT;
SET GTID_NEXT="AUTOMATIC";
START SLAVE;
```

Tras la corrección, la validación final mediante `SHOW SLAVE STATUS\G` confirmó la sincronización exitosa de ambos hilos:

- `Slave_IO_Running: Yes`
- `Slave_SQL_Running: Yes`

8.4. Cargar base de datos ECOBICI

El contenedor mysql-master es un sistema operativo Linux aislado. No tiene disco C: ni identifica la ruta:

```
C:\ProgramData\MySQL\MySQL Server 8.0\Uploads.
```

para lo LOAD DATA INFILE del script.

Pasaremos los archivos de Windows al entorno de Linux en Docker y se modificaran las rutas.

8.4.1. Copiar los archivos CSV al contenedor Linux

```
docker cp "/mnt/c/ProgramData/MySQL/MySQL Server 8.0/Uploads/." mysql-master:/var/
lib/mysql-files/
```

8.4.2. Mover y modificar el script SQL

Mover el archivo `QueryTS_Ecobici_CDMX.sql` a la carpeta del proyecto en Linux.

```
\\wsl$\Ubuntu\home\ricardo\proyecto_ecobici.
```

```
Buscar y Reemplazar
Buscar
C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/
Reemplazar
/var/lib/mysql-files/
```

8.4.3. Inyectar la base de datos

En la terminal de Linux ejecuta el comando para inyectar todo el script directamente al maestro:

```
docker exec -i mysql-master mysql -uroot -p_xxxx DB_Ecobici_CDMX <
QueryTS_Ecobici_CDMX.sql
```

Importante

Como este script carga más de 6 millones de registros y construye llaves foráneas e índices, el comando se quedará pensando durante unos 15 a 20 minutos. No lo canceles.

Si quieres ver cómo tu computadora está trabajando, puedes abrir una nueva pestaña o ventana de terminal en WSL y ejecutar el comando. Ahí verás en tiempo real cómo el contenedor mysql-master está utilizando al máximo tu CPU y memoria RAM.

```
docker stats.
```

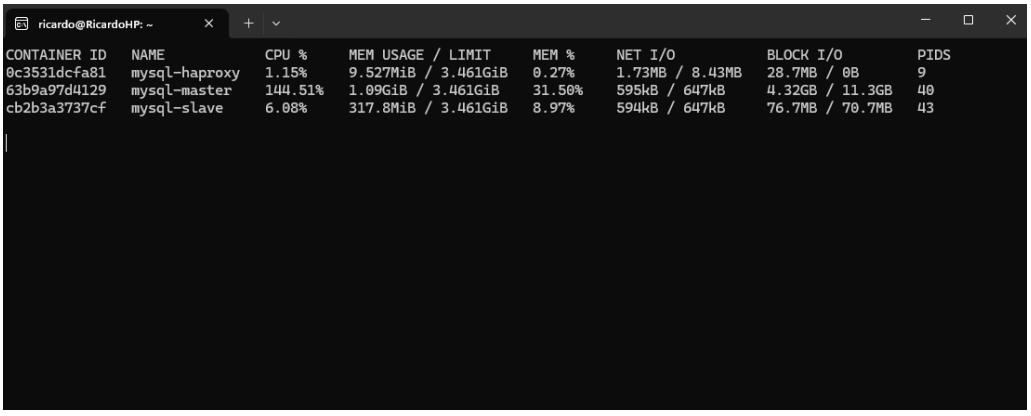


Figura 7: Estatus carga

9. Crear Cuenta Google Cloud GCP.

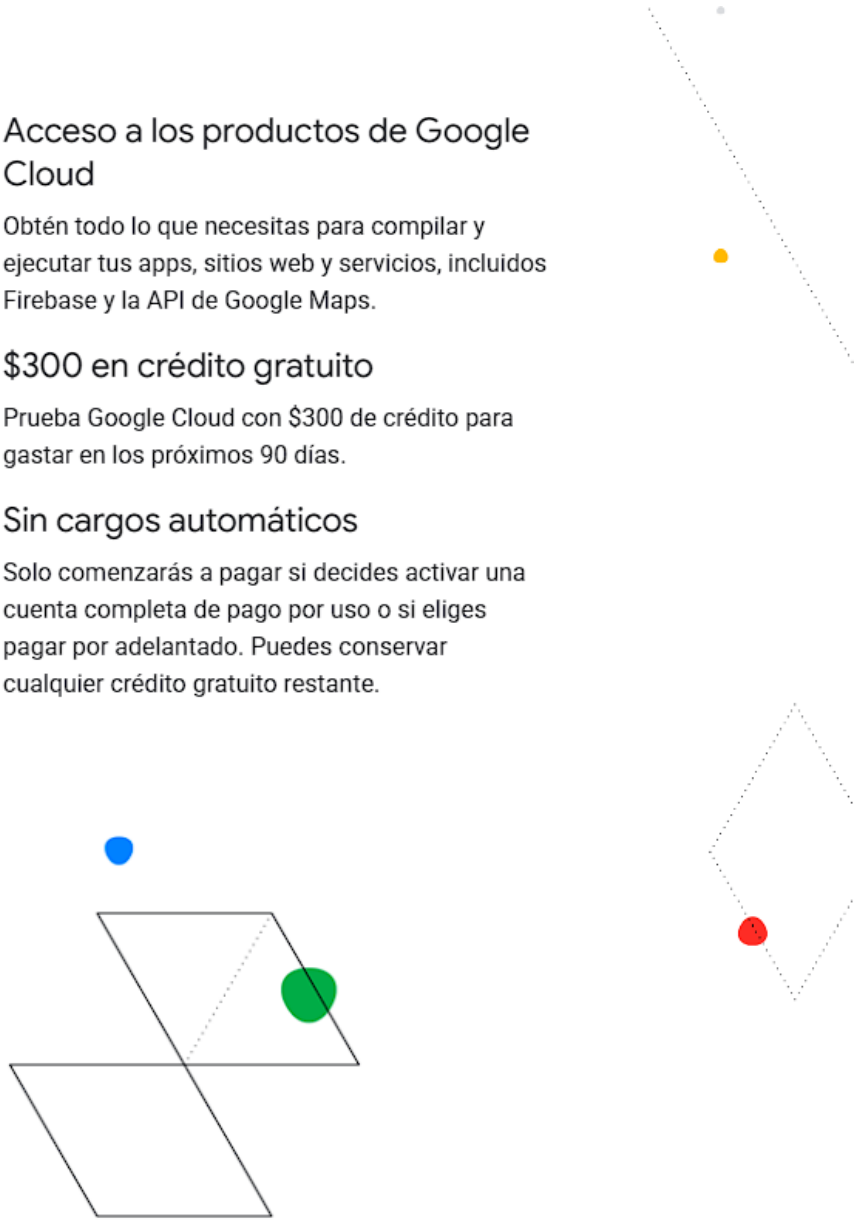


Figura 8: GCP

9.1. Crear Instancias

- Ingresa a la Consola de Google Cloud.
- En el menú de navegación ve a Compute Engine >Instancias de VM.
- Haz clic en Crear instancia (Create instance).
- Configura los siguientes parámetros: Nombre: nodo-respaldo-ecobici Región: Selecciona us-central1 (Iowa) Tipo de máquina: Propósito general, serie E2, selecciona e2-micro (Es la opción más barata).

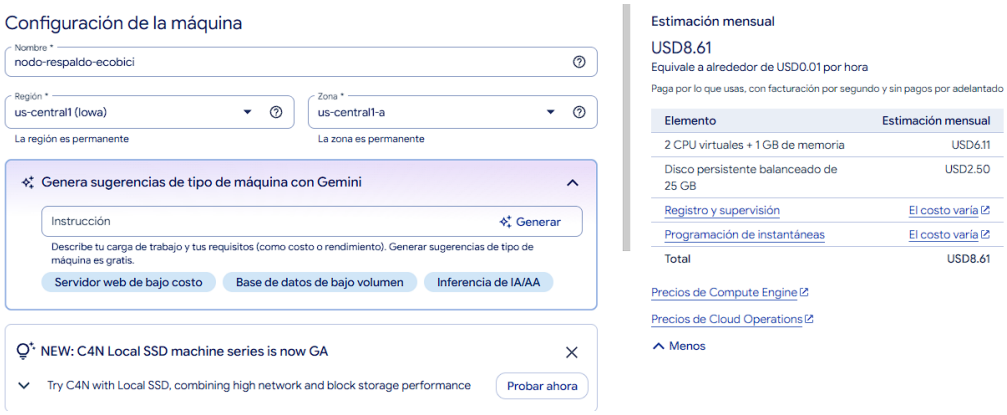


Figura 9: Cofg mv

- Disco de arranque (Boot disk): Haz clic en Cambiar. Sistema Operativo: Selecciona Ubuntu. Versión: Selecciona Ubuntu 24.04 LTS.

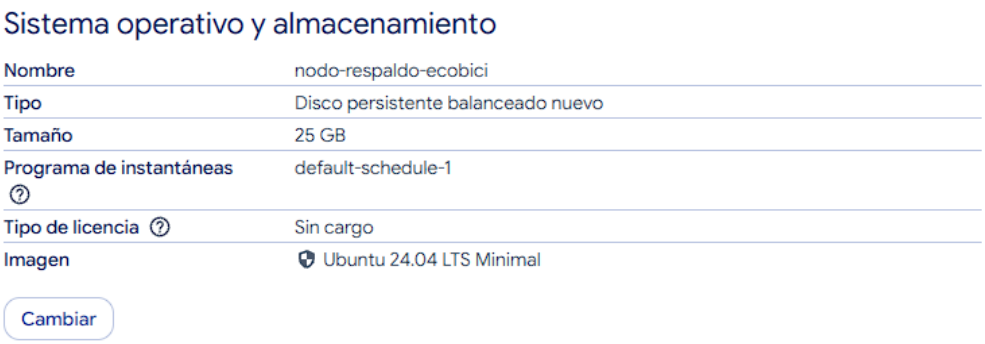


Figura 10: Cof SO

Tamaño: 10 GB (por defecto). Firewall: Marca la casilla Permitir tráfico HTTP y Permitir tráfico HTTPS.

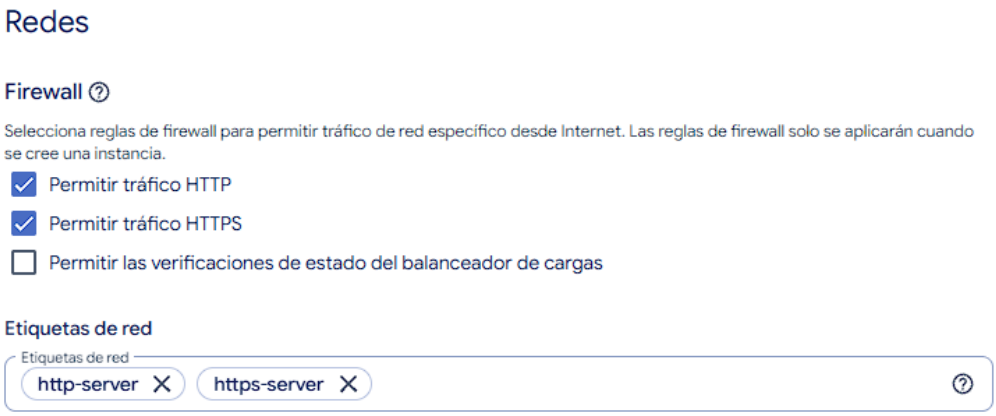


Figura 11: Firewall

Haz clic en el botón azul Crear al final de la página.

9.1.1. Regla de Firewall adicional

Las casillas HTTP/HTTPS que se marcaron abren los puertos 80 y 443. Pero necesitamos abrir los puertos específicos de la bases de datos (3306 y 8404).

- En el submenú que se despliega, haz clic en Firewall.
- En la parte superior central de la pantalla de Firewall, busca y haz clic en el botón que dice CREAR REGLA DE FIREWALL (Create Firewall Rule).
- Nombre: allow-haproxy
- Dirección del tráfico: Entrada
- Acción: Permitir
- Destinos: Todas las instancias de la red
- Rangos IPv4 de origen: 0.0.0.0/0
- Puertos TCP: 3306,8404

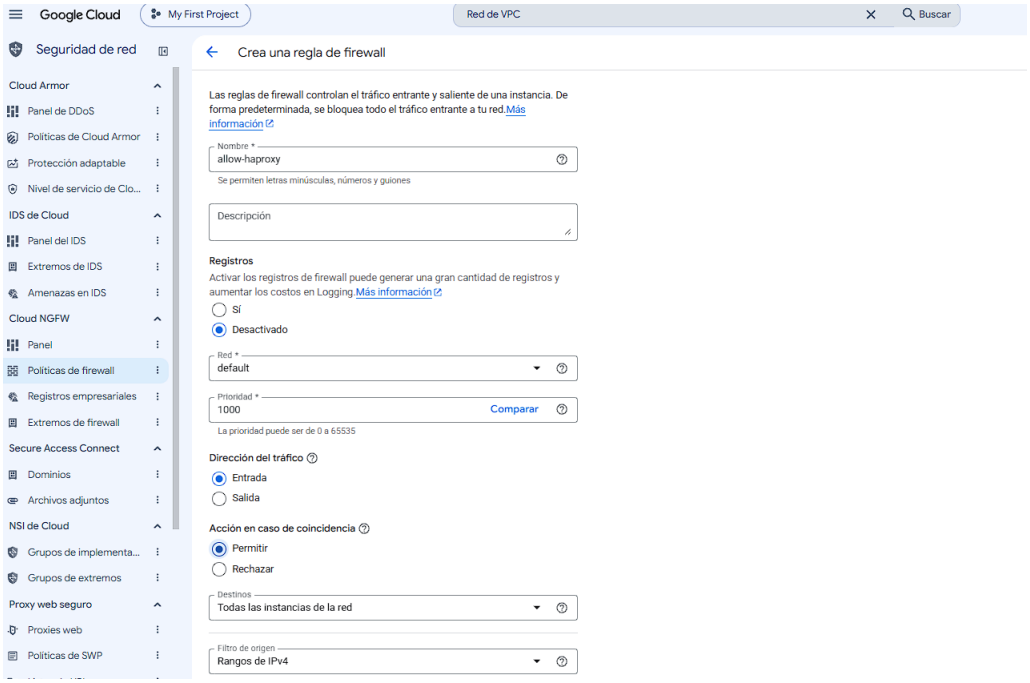


Figura 12: Firewall

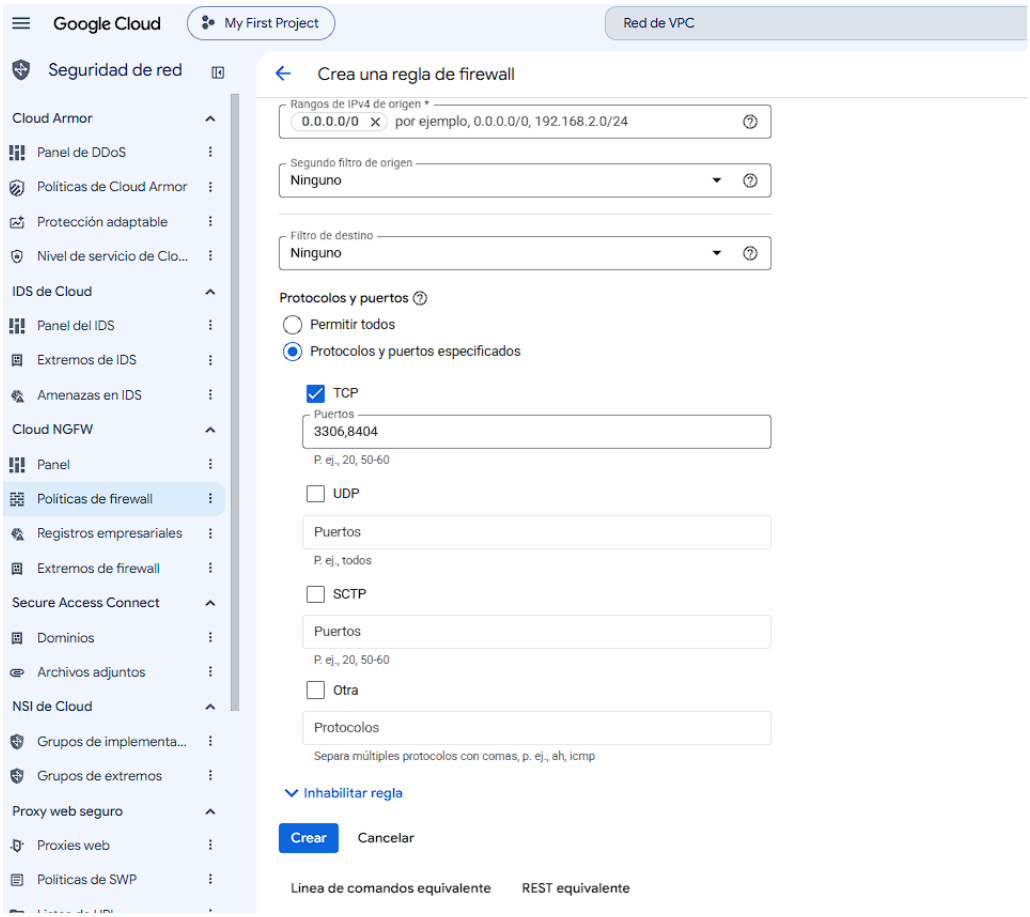


Figura 13: Firewall

10. Maquina Virtual Cloud GCP.

Maquina Virtual: Se observa la terminal de Ubuntu 2404 LTS conectada vá SSH

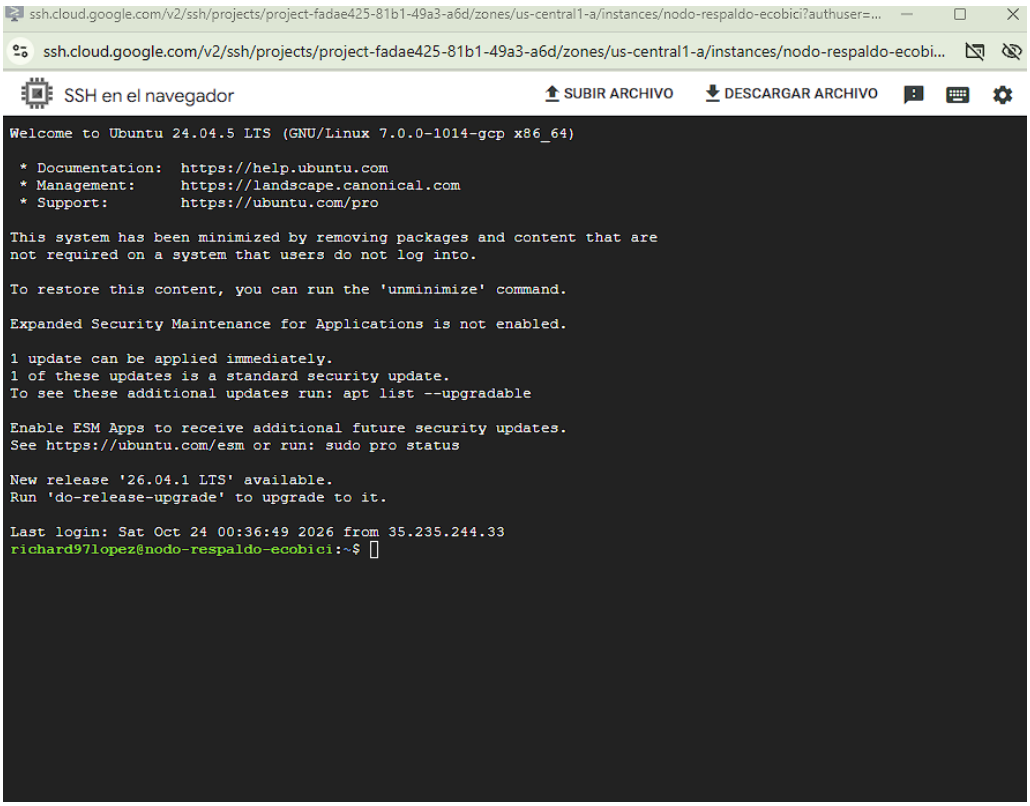


Figura 14: Firewall

11. Instalar Docker y Docker Compose Cloud GCP.

Se instalara Docker en la maquina virtual y se caragra el proyecto
Primero actualizaremos los paquetes del sistema y luego instalaremos el motor de Docker.

```
sudo apt update && sudo apt upgrade -y
sudo apt install docker.io docker-compose-v2 -y
```

11.1. Dar permisos a tu usuario

Para no tener que escribir sudo antes de cada comando de Docker, añadiremos tu usuario al grupo de Docker:

```
sudo usermod -aG docker $USER
newgrp docker
```

11.1.1. Crear la estructura del proyecto

Crearemos las mismas carpetas que estan el localhost.

```
mkdir -p ~/proyecto_ecobici/haproxy
cd ~/proyecto_ecobici
```

11.1.2. Instalacion editor de texto nano

```
sudo apt install nano -y
```

11.1.3. Crear docker-compose.yml

Arquitectura con GTID inyectado.

```
nano docker-compose.yml
```

11.1.4. Crear haproxy.cfg

La configuración del balanceador de carga dentro de su respectiva carpeta.

```
nano haproxy/haproxy.cfg
```

12. Arrancar la infraestructura en la nube Cloud GCP.

Con los archivos listos, ejecuta este comando para descargar las imágenes y encender los servidores de base de datos.

```
docker compose up -d
```

La arquitectura esta completa.

- Maestro
- Esclavo
- HAProxy

Estan levantados los contenedores y de forma estable en la maquina virtual de GCP.

12.1. Configuración de Permisos para HAProxy (Health Checks)

La infraestructura funciona, los nodos de MySQL rechazan la conexión del balanceador porque todavía no se crea el usuario que HAProxy usa para verificar la salud de los servidores haproxy check.
Para el Maestro:

```
docker exec -it mysql-master mysql -uroot -p_ -e "DROP USER IF EXISTS 'haproxy_check
'@'%'; CREATE USER 'haproxy_check'@'%' IDENTIFIED WITH mysql_native_password BY '';
FLUSH PRIVILEGES;"
```

Para la Réplica:

```
docker exec -it mysql-slave mysql -uroot -p_ -e "DROP USER IF EXISTS 'haproxy_check'@
'%'; CREATE USER 'haproxy_check'@'%' IDENTIFIED WITH mysql_native_password BY '';
FLUSH PRIVILEGES;"
```

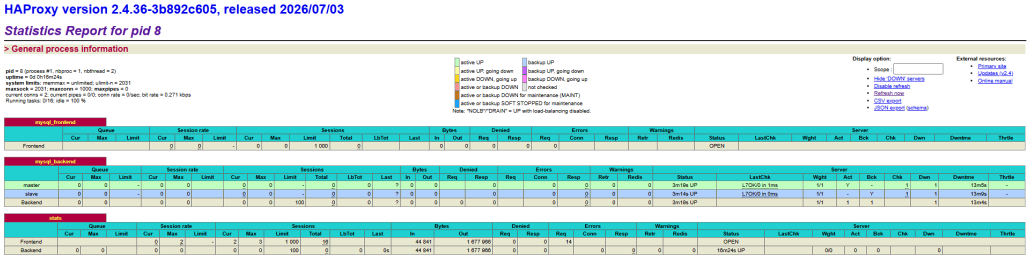


Figura 15: HAProxy

12.1.1. Replicacion GTID

IP interna del Maestro

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mysql-
 master
```

12.1.2. Crear el usuario de clonación en el Maestro

Entra a la terminal del Maestro:

```
docker exec -it mysql-master mysql -uroot -p_XXX
```

MySQL.

```
 CREATE USER 'replica_user'@'%' IDENTIFIED WITH mysql_native_password BY 'xxx';
 GRANT REPLICATION SLAVE ON *.* TO 'replica_user'@'%';
 FLUSH PRIVILEGES;
 EXIT;
```

Conectar la Réplica al Maestro:

```
docker exec -it mysql-slave mysql -uroot -p_xxxx
```

MySQL.

```
 STOP SLAVE;

 CHANGE MASTER TO
 MASTER_HOST='mysql-master',
 MASTER_USER='replica_user',
 MASTER_PASSWORD='xxxx',
 MASTER_AUTO_POSITION=1;

 START SLAVE;

 SHOW SLAVE STATUS\G
```

12.1.3. Comprobar la conexión

```
SHOW SLAVE STATUS\G
```

La replicación asíncrona por GTID está conectada y funcionando de forma estable.

Se contruyo, desplego y estabilizo una arquitectura de Alta Disponibilidad Master - Slave con balanceador de carga en HAProxy en GCP.

13. Poblar la Base de Datos Cloud GCP.

Necesitas enviar el respaldo de tu esquema relacional local a la nube. abriendo MySQL Workbench, localhost y creando una nueva conexión hacia tu instancia en GCP.

- Host: 34.122.XXXX
- Puerto: XXXX.
- Contraseña: XXXX.

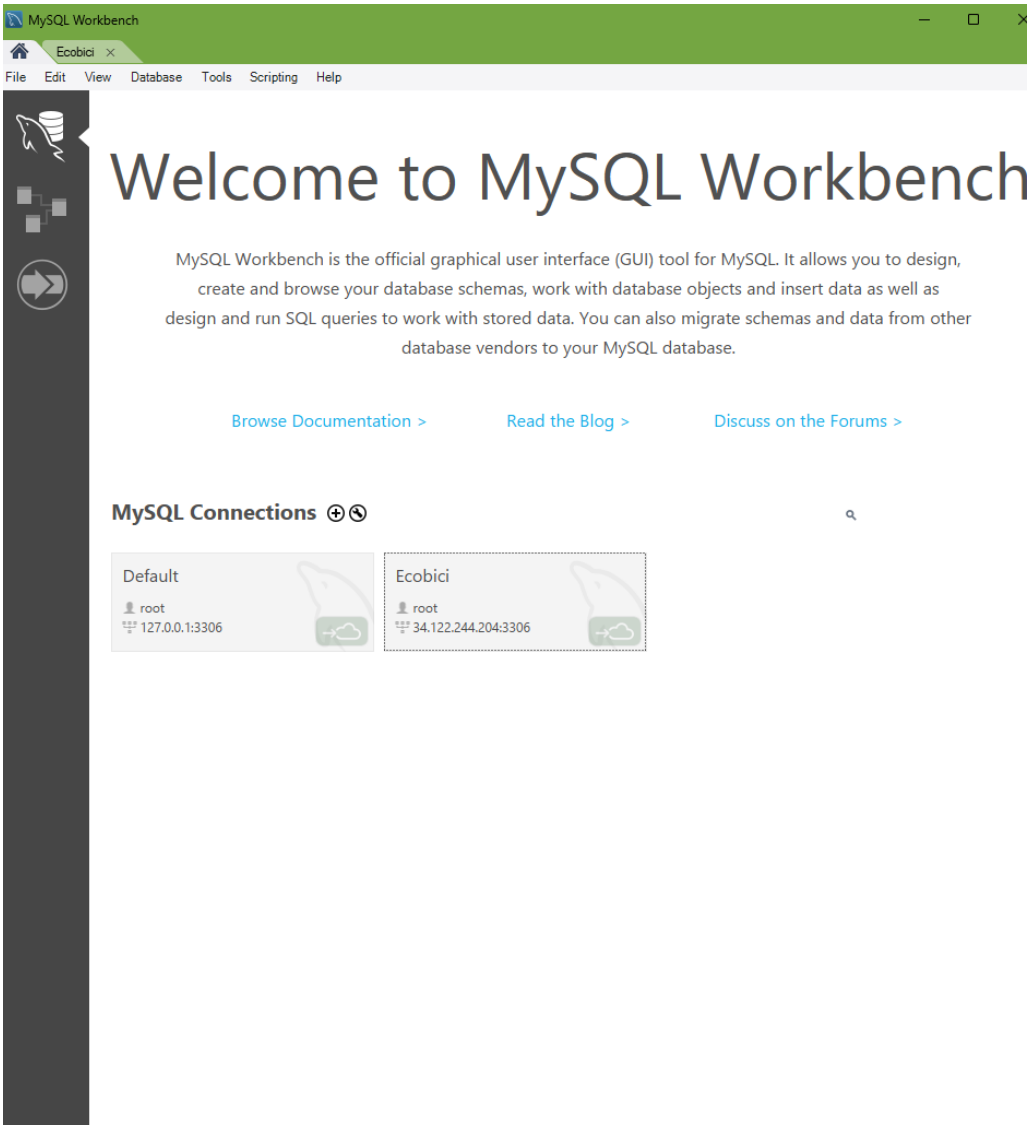


Figura 16: HAProxy

13.1. Cargar Data en la Maquina Virtual GCP

Cargar la Data de Ecobici suben 12 archivos CSV a la nube.

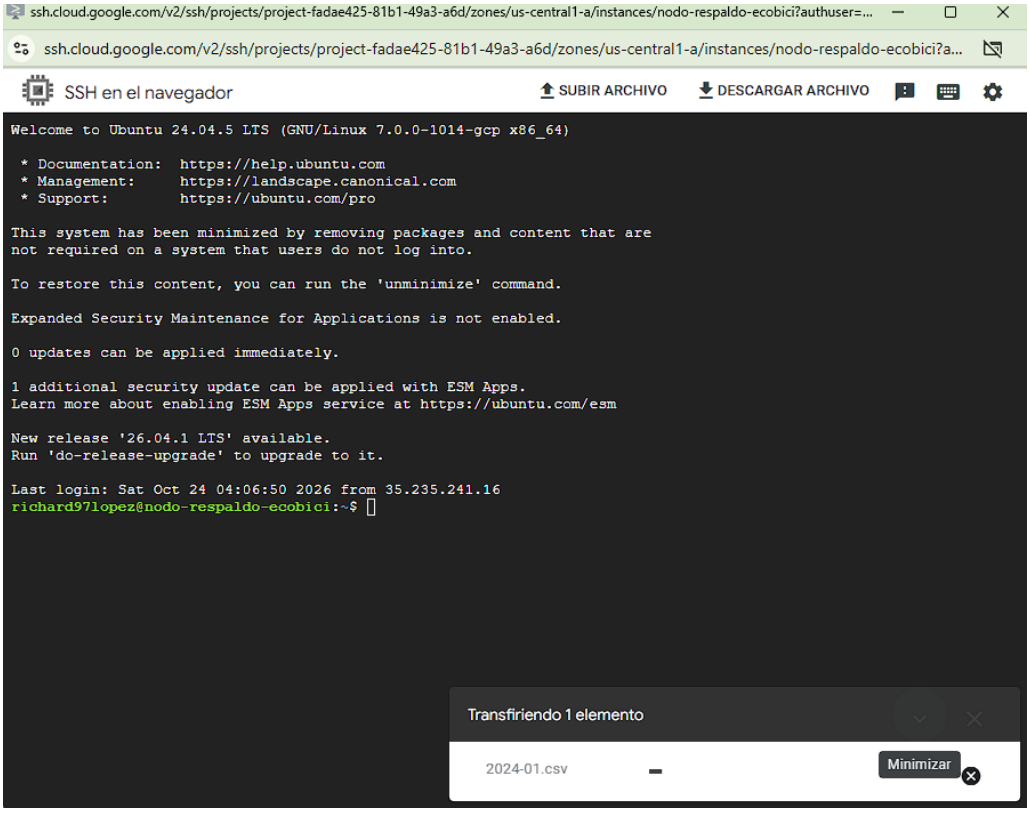


Figura 17: Data

13.1.1. Copiar dentro del contenedor Maestro

Sedeben copiar dentro del contenedor Maestro usando el comando en tu terminal SSH:

```
docker cp *.csv mysql-master:/var/lib/mysql-files/
```

Finalmente, en MySQL Workbench, modifica todas las rutas de los archivos en tu script.

```
Cambia
C:/ProgramData/MySQL/
por
/var/lib/mysql-files/nombre_del_archivo.csv.
```

Los 12 Archios CSV ya están alojados dentro del nodo Maestro en la ruta

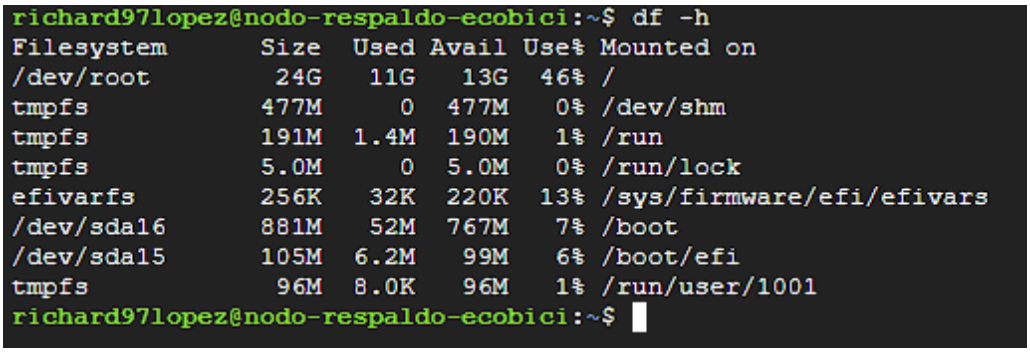


Figura 18: VM

13.1.2. Ejecución del Script en MySQL Workbench

- Script QueryTS Ecobici CDMX en MySQL Workbench.
- Herramienta de Buscar y Reemplazar.
- Escribe la ruta exacta de Windows que tiene tu script actualmente.

```
        C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/

```

- Replace, escribe la nueva ruta de Linux.

```
        /var/lib/mysql-files/

```

- Ejecutar todo el script

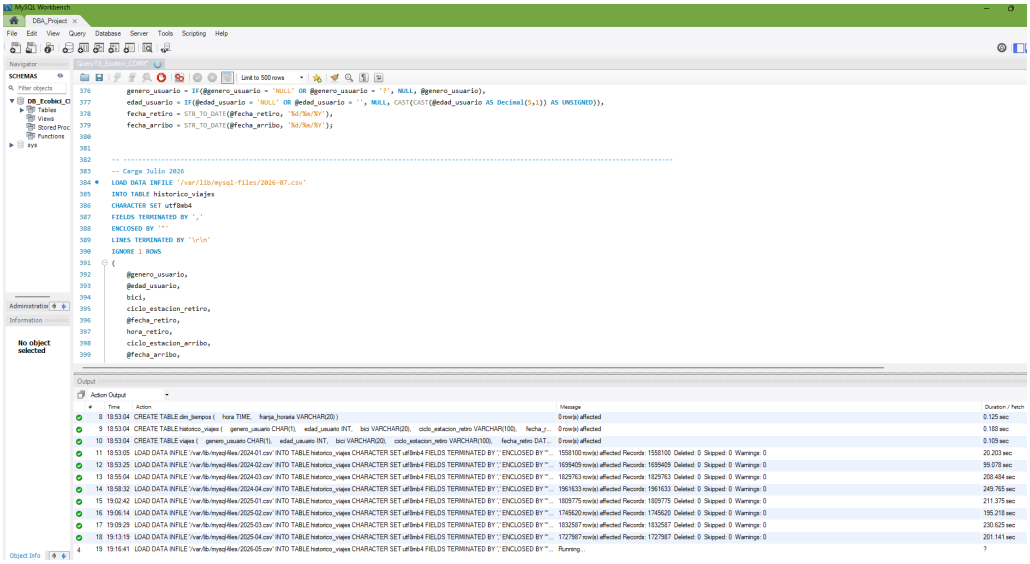


Figura 19: VM

13.1.3. PRIMARY KEY

Se agregan las llaves primarias.

```
    ALTER TABLE cat_generos ADD PRIMARY KEY (codigo_genero);
    ALTER TABLE estaciones ADD PRIMARY KEY (id_estacion);
    ALTER TABLE bicicletas ADD PRIMARY KEY (bici);
    ALTER TABLE dim_usuarios ADD PRIMARY KEY (id_perfil);
    ALTER TABLE dim_fechas ADD PRIMARY KEY (fecha);
    ALTER TABLE dim_tiempos ADD PRIMARY KEY (hora);
```

13.1.4. FOREIGN KEY

```
    ALTER TABLE viajes
        ADD CONSTRAINT fk_viajes_genero FOREIGN KEY (genero_usuario) REFERENCES
    cat_generos(codigo_genero),
        ADD CONSTRAINT fk_viajes_bici FOREIGN KEY (bici) REFERENCES bicicletas(bici),
        ADD CONSTRAINT fk_viajes_estacion_retiro FOREIGN KEY (ciclo_estacion_retiro)
    REFERENCES estaciones(id_estacion),
        ADD CONSTRAINT fk_viajes_estacion_arribo FOREIGN KEY (ciclo_estacion_arribo)
    REFERENCES estaciones(id_estacion),
        ADD CONSTRAINT fk_viajes_fecha_retiro FOREIGN KEY (fecha_retiro) REFERENCES
    dim_fechas(fecha),
        ADD CONSTRAINT fk_viajes_hora_retiro FOREIGN KEY (hora_retiro) REFERENCES
    dim_tiempos(hora);
```

14. Configuracion de Parametros Cloud GCP.

Tu instancia nodo-respaldo-ecobici tiene 1 GB de RAM total. La regla del 60 - 80 asume que hay un único servidor de base de datos instalado en el sistema operativo. Dado que tu entorno de Alta Disponibilidad corre dos contenedores MySQL (Maestro y Esclavo) junto con HAProxy en la misma máquina virtual.

Se asignaran 300MB a cada nodo, lo que en conjunto representara el 60 de la RAM de la maquina virtual.

Se crean las carpetas.

```
mkdir -p ~/proyecto_ecobici/master/conf/mysql.conf.d/
mkdir -p ~/proyecto_ecobici/slave/conf/mysql.conf.d/
```

14.1. Configurar el nodo Maestro

```
nano ~/proyecto_ecobici/master/conf/mysql.conf.d/mysqld.cnf

innodb_buffer_pool_size = 300M
innodb_log_buffer_size = 32M
```

14.2. Configurar el nodo Esclavo

```
nano ~/proyecto_ecobici/slave/conf/mysql.conf.d/mysqld.cnf

innodb_buffer_pool_size = 300M
innodb_log_buffer_size = 32M
o
echo -e "[mysqld]\ninnodb_buffer_pool_size = 300M\ninnodb_log_buffer_size = 32M" >
~/proyecto_ecobici/master/conf/mysql.conf.d/mysqld.cnf
```

14.3. Aplicar los cambios en el clúster

```
cd ~/proyecto_ecobici
docker compose restart mysql-master mysql-slave

o
echo -e "[mysqld]\ninnodb_buffer_pool_size = 300M\ninnodb_log_buffer_size = 32M" >
~/proyecto_ecobici/slave/conf/mysql.conf.d/mysqld.cnf
```

Verificar.

```
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
```

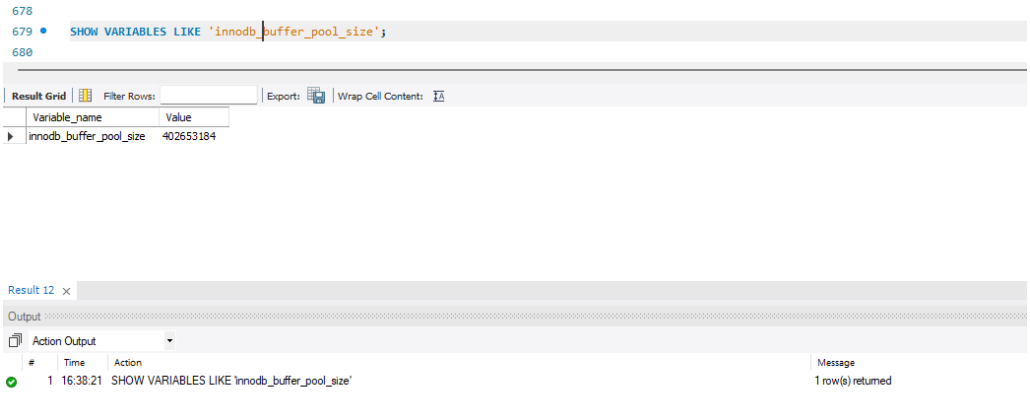


Figura 20: buffer pool

15. Disponibilidad y Automatizacion Cloud GCP.

15.1. Instalar la herramienta cron

```
sudo apt update && sudo apt install cron -y
```

```
sudo systemctl enable --now cron
```

Crear un script de Bah en el servidor Ubuntu que ejecute

```
    mysqldump
con la bandera
    --single-transaction
```

Para no bloquear escrituras durante el respaldo y se automatizara con cron para que se ejecute solo en la madrugada.

15.2. Crear la carpeta y el script de respaldo

```
    mkdir ~/proyecto_ecobici/backups
    nano ~/proyecto_ecobici/backup.sh
```

15.2.1. Script de automatización

–source-data=2

```
    !/bin/bash
    Script de Respaldo Logico para Ecobici CDMX

    FECHA=$(date +"%Y-%m-%d_%H-%M")
    DESTINO="/home/richard97lopez/proyecto_ecobici/backups/ecobici_full_$FECHA.sql"

    1. Ejecutar mysqldump a traves del contenedor Maestro
    docker exec mysql-master mysqldump -uroot -pxxx --single-transaction --source-data=2
     --routines --events --triggers DB_Ecobici_CDMX > $DESTINO

    2. Comprimir el respaldo para no saturar el disco de 10 GB de GCP
    gzip $DESTINO
```

15.2.2. Dar permisos y programar la tarea automática

El script sea ejecutable.

```
    chmod +x ~/proyecto_ecobici/backup.sh
```

```
    crontab -e
```

Si te pregunta qué editor usar, presiona 1 para nano

Al final del archivo y indica en la línea a Linux que ejecute tu script todos los días a las 2:00 AM hora del servidor.

```
    0 2 * * * /home/richard97lopez/proyecto_ecobici/backup.sh
```

15.2.3. Comprobar el peso del respaldo

Para verificar físicamente que el script que acabas de ejecutar funcionó y comprimió tus 6.5 millones de viajes.

```
    ls -lh ~/proyecto_ecobici/backups/
```

15.3. Respaldo Incremental Diario (Binlogs)

El script copiará la bitácora de transacciones (binlogs) todos los días, permitiendo recuperar la base de datos hasta un segundo exacto antes de un accidente (Point-In-Time Recovery).

```
    mkdir -p ~/proyecto_ecobici/backups/binlogs
    mkdir -p ~/proyecto_ecobici/logs
```

```
    nano ~/proyecto_ecobici/respaldo_binlogs.sh
```

```
    !/bin/bash
Respaldo incremental diario de binlogs
DIR=$HOME/proyecto_ecobici
DESTINO=$DIR/backups/binlogs
MYSQL="docker exec mysql-master mysql -uroot -p_xxx-N -e"

1. Rotacion: cerrar el binlog actual y abrir uno nuevo
$MYSQL "FLUSH BINARY LOGS;"

2. Identificar el binlog activo (no se copia)
ACTIVO=$($MYSQL "SHOW MASTER STATUS;" | awk '{print $1}')

3. Copiar los binlogs cerrados que aun no esten respaldados
for f in $(docker exec mysql-master bash -c "ls /var/lib/mysql | grep -E '^mysql-bin
    \.[0-9]+$'"); do
    if [ "$f" != "$ACTIVO" ] && [ ! -f "$DESTINO/$f" ]; then
        docker cp mysql-master:/var/lib/mysql/$f $DESTINO/
        echo "$(date) Binlog copiado: $f" >> $DIR/logs/respaldos.log
    fi
done

4. Conservar 14 dias de binlogs respaldados
find $DESTINO -name "mysql-bin.*" -mtime +14 -delete
```

```
    chmod +x ~/proyecto_ecobici/respaldo_binlogs.sh
```

15.4. Mantenimiento Preventivo Automatizado ANALYZE y OPTIMIZE

Para desfragmentar la base de datos limpiar los huecos que dejan los DELETE o UPDATE, se usara el programador interno de MySQL (Event Scheduler).

15.4.1. MySQL Workbench

En una nueva pestaña de consultas SQL se ejecuta el código para crear el evento.

```
1   USE DB_Ecobici_CDMX;
2
3   DELIMITER $$
4   CREATE EVENT ev_mantenimiento_semanal
5   ON SCHEDULE EVERY 1 WEEK
6   STARTS CURRENT_TIMESTAMP
7   COMMENT 'ANALYZE semanal y OPTIMIZE mensual (primer sabado)'
8   DO
9   BEGIN
10      ANALYZE TABLE viajes, historico_viajes;
11      IF DAYOFMONTH(CURRENT_DATE) <= 7 THEN
12          OPTIMIZE TABLE viajes;
13      END IF;
14  END$$
15  DELIMITER ;
16
17  SELECT event_name, status, last_executed FROM information_schema.events WHERE
    event_schema = 'DB_Ecobici_CDMX';
```

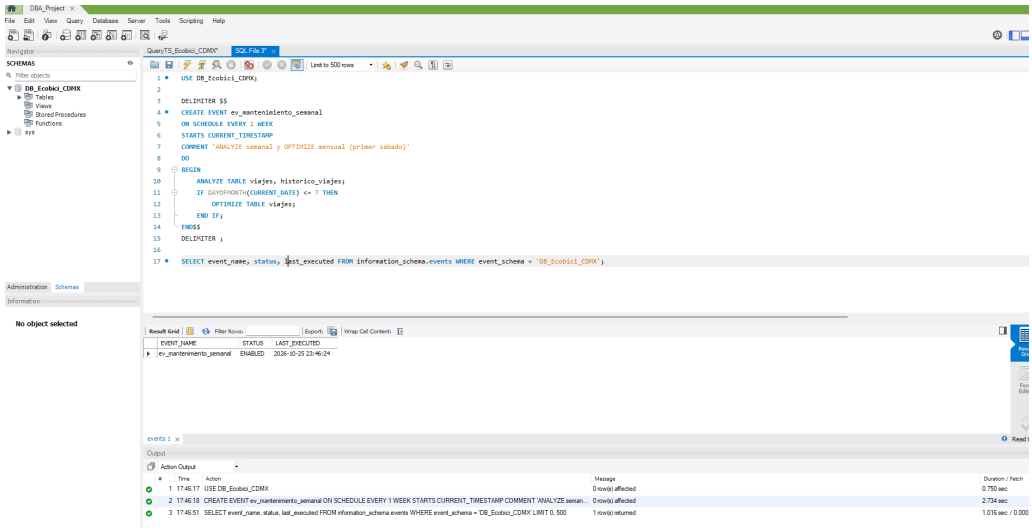


Figura 21: CREATE EVENT

15.5. Rotación de Particiones y Retención de Datos

La tabla histórica archive los datos viejos y que la tabla activa no crezca de forma descontrolada. Se Crea el script de rotación en tu terminal

```
nano ~/proyecto_ecobici/rotacion_particiones.sh
```

```
!/bin/bash
Rotacion mensual de particiones y generacion de resumen
DIR=$HOME/proyecto_ecobici
MYSQL="docker exec mysql-master mysql -uroot -p_xxx -DB_Ecobici_CDMX -N -e"

Obtenemos el mes anterior (el que vamos a rotar)
MES_A_ROTAR=$(date -d "last month" +"%Y%m")
PARTICION="p_$MES_A_ROTAR"

1. Resumir los datos antes de borrarlos
$MYSQL "INSERT IGNORE INTO resumen_viajes_mes (anio, mes, ciclo_estacion_retiro, genero_usuario, total_viajes, edad_promedio, duracion_promedio_min)
SELECT YEAR(fecha_retiro), MONTH(fecha_retiro), ciclo_estacion_retiro, IFNULL(genero_usuario, 'N'), COUNT(*), AVG(edad_usuario), AVG(TIME_TO_SEC(TIMEDIFF(hora_arribo, hora_retiro))/60)
FROM historico_viajes PARTITION ($PARTICION)
GROUP BY 1, 2, 3, 4;"

2. Eliminar el detalle pesado
$MYSQL "ALTER TABLE historico_viajes DROP PARTITION $PARTICION;"

echo "$(date) Rotacion completada para $PARTICION" >> $DIR/logs/respaldos.log
```

```
chmod +x ~/proyecto_ecobici/rotacion_particiones.sh
```

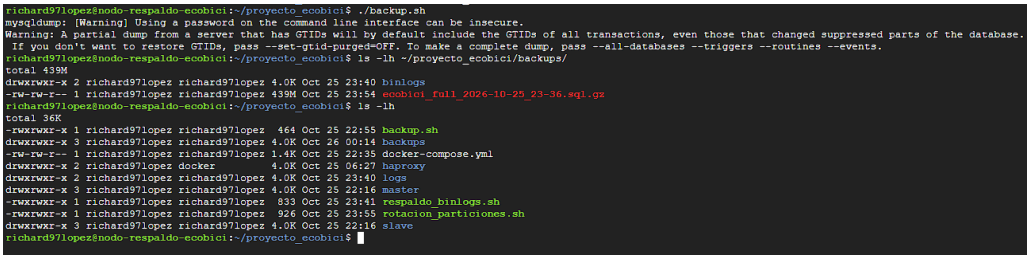


Figura 22: Pogramador de tareas

15.6. Programación de tareas(Cron) y Monitoreo

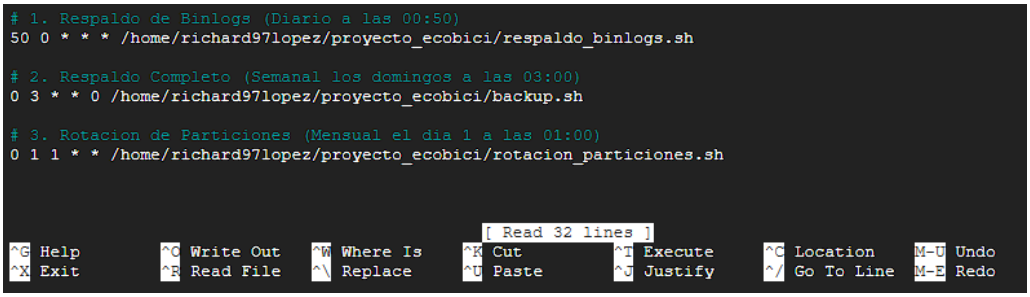


Figura 23: Pogramador de tareas

16. Plan de Recuperación ante Desastres (DRP)

16.1. Alcance y Objetivo del DRP

El objetivo principal de este DRP es garantizar la continuidad operativa del sistema de Ecobici CDMX ante cualquier eventualidad técnica o desastre. Se establecen dos métricas clave para la estrategia de recuperación:

- **RTO (Recovery Time Objective):** Minimizar el tiempo de inactividad del servicio a tan solo unos segundos, logrando la conmutación automática del tráfico mediante los *health checks* de HAProxy hacia los nodos secundarios.
- **RPO (Recovery Point Objective):** Garantizar una pérdida de datos cercana a cero, combinando la replicación síncrona/asíncrona activa con una restauración exacta en el tiempo (PITR) apoyada en bitácoras de transacciones.

16.2. Arquitectura de Respaldo Existente

La infraestructura en Google Cloud Platform está protegida por un esquema de copias de seguridad automatizadas mediante `cron`:

- **Respaldo Lógico Semanal:** Se ejecuta todos los domingos a las 03:00 hrs. Utiliza `mysqldump` con la directiva `-source-data=2` para incluir las coordenadas de replicación GTID. Este proceso genera un archivo comprimido de aproximadamente 439 MB sin bloquear la operación gracias a `-single-transaction`.
- **Respaldos Incrementales (Binlogs):** Un script automatizado se ejecuta diariamente a las 00:50 hrs para rotar (`FLUSH BINARY LOGS`) y exportar la bitácora de transacciones del día, manteniendo una política de retención estricta de 14 días.

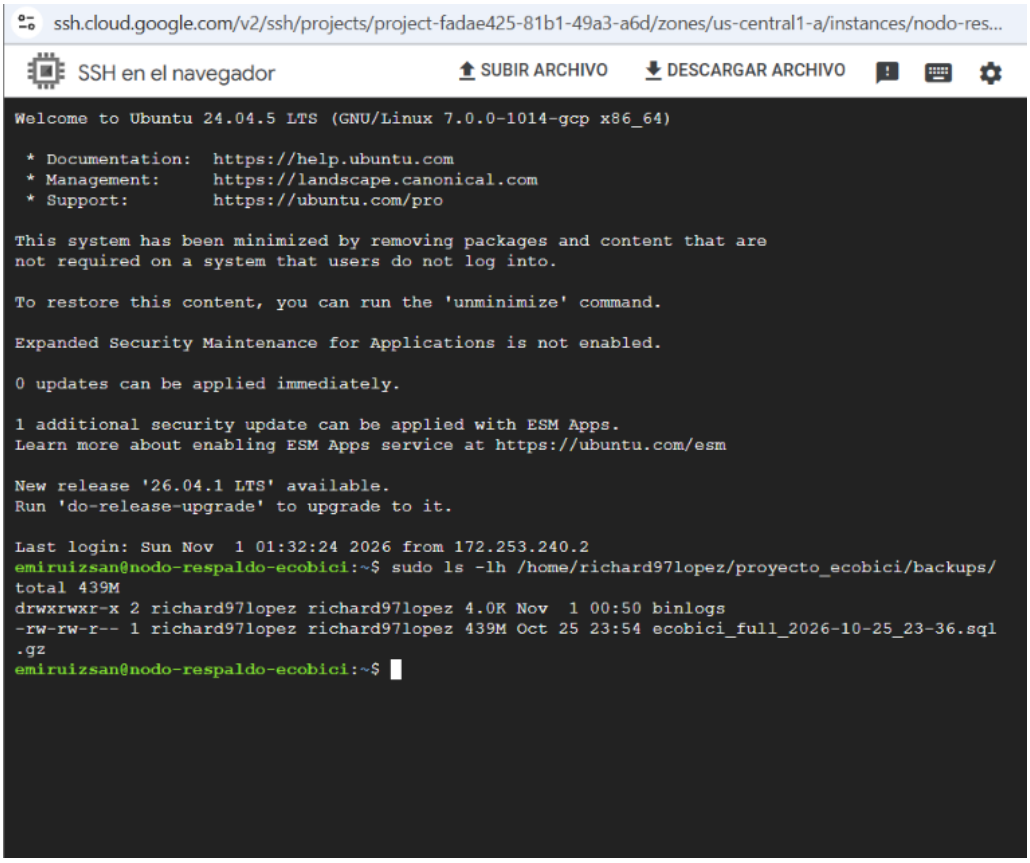


Figura 24: Evidencia del respaldo lógico completo automatizado en GCP.

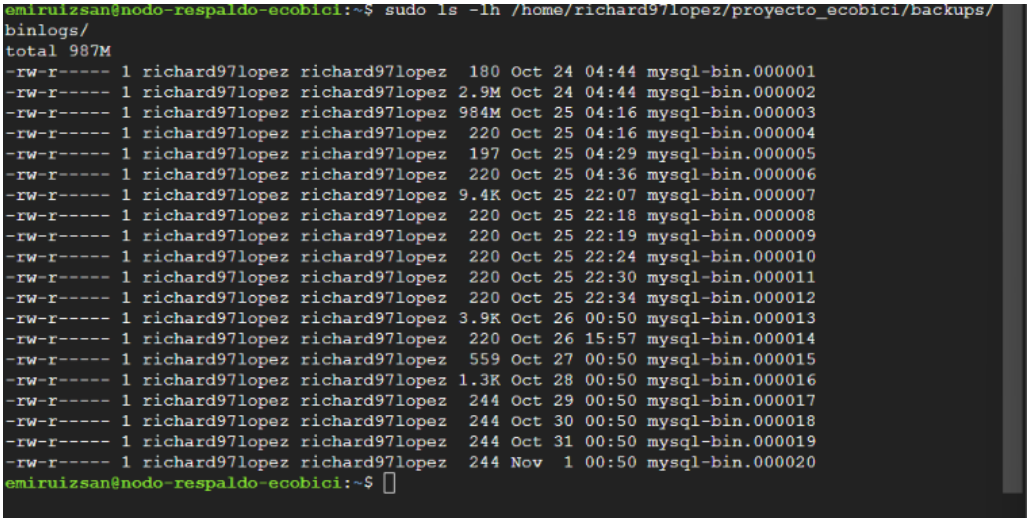


Figura 25: Evidencia de la bitácora de transacciones (Binlogs) para recuperación PITR.

16.3. Protocolo de Respuesta (Escenarios de Desastre)

Caso 1: Caída del Servidor Primario (Hard Crash)

Si el nodo Maestro sufre un apagón, pérdida total de conectividad o falla de hardware en GCP, el mecanismo de respuesta es el siguiente:

1. **Detección y Redirección:** HAProxy detecta la caída del Maestro y redirige el tráfico automáticamente hacia la Réplica.
2. **Modo de Protección:** La Réplica recibe el tráfico en modo `read-only`, permitiendo consultas pero protegiendo la consistencia de los datos.
3. **Promoción a Maestro:** Si el nodo original es irrecuperable, el administrador interviene deshabilitando el modo lectura en la Réplica (`SET GLOBAL read_only = OFF;`) para promoverla a Maestro principal y aceptar nuevas escrituras.

Caso 2: Corrupción del Disco o Errores DML (Tablespace `viajes`)

Ante una eliminación accidental masiva (`DELETE` sin `WHERE`) o corrupción del archivo `.ibd`:

1. **Aislamiento:** Se detiene inmediatamente la replicación en el nodo sano (`STOP SLAVE;`) para evitar que la corrupción se propague por GTID.
2. **Restauración Base:** En una instancia aislada de recuperación, se restaura el último Respaldo Completo de 439 MB.
3. **PITR (Point-In-Time Recovery):** Se aplican secuencialmente los Binlogs utilizando `mysqlbinlog` con el parámetro `-stop-datetime`, deteniendo la inyección de eventos exactamente un segundo antes del error. Una vez validada, la data limpia se reintegra a producción.

Caso 3: Mantenimiento Preventivo (Switchover)

Para realizar actualizaciones del sistema operativo o escalar el hardware de la máquina virtual sin interrumpir el servicio:

1. **Bloqueo de Escrituras:** Se configura el Maestro en modo lectura estricta (`SET GLOBAL super_read_only = ON;`).
2. **Sincronización GTID:** Se monitorea la Réplica hasta confirmar que ha procesado la última transacción pendiente del Maestro.
3. **Conmutación (Failover Controlado):** Se apaga el servicio MySQL del Maestro ordenadamente. HAProxy detectará la baja controlada y asignará el tráfico de la aplicación a la Réplica mientras dura la ventana de mantenimiento.

17. Pagina web

```
http://34.136.125.78:8404/stats
https://app.sendgrid.com/
https://console.cloud.google.com/
https://red-bicicletas-rb.onrender.com
```

17.1. Backend y Frontend

Repositorio Bitbucket

Clonar Repositorio Bitbucket

```
git clone https://richard-lprz-admin@bitbucket.org/richard-lprz/red_bicicletas_rb.git
```



Figura 26: Pagina principal



Figura 27: Pagina principal



Figura 28: Pagina principal.

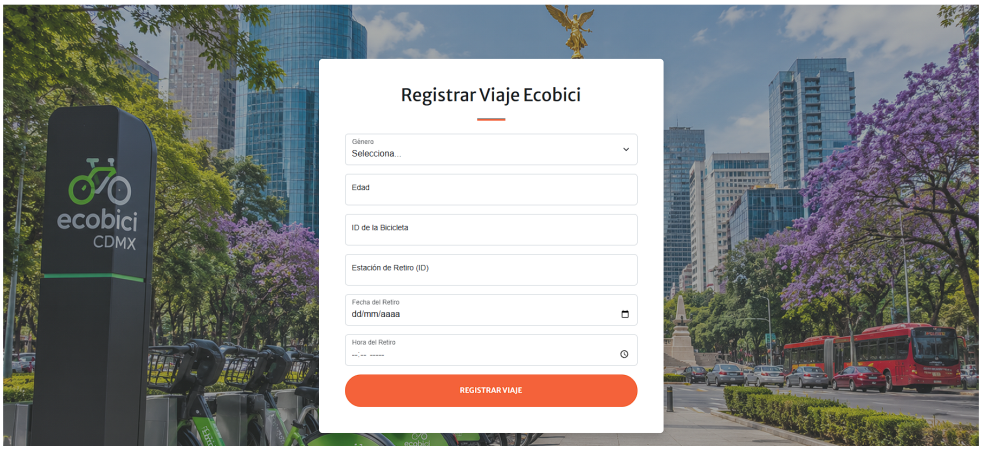


Figura 29: Pagina principal para registro

18. Servidor Render

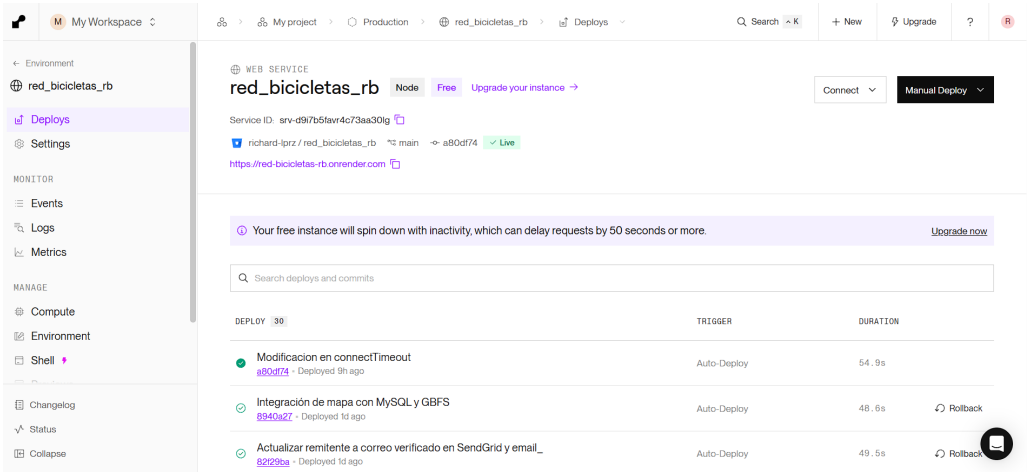


Figura 30: Web Services

19. Plataforma de envío de correos electrónico

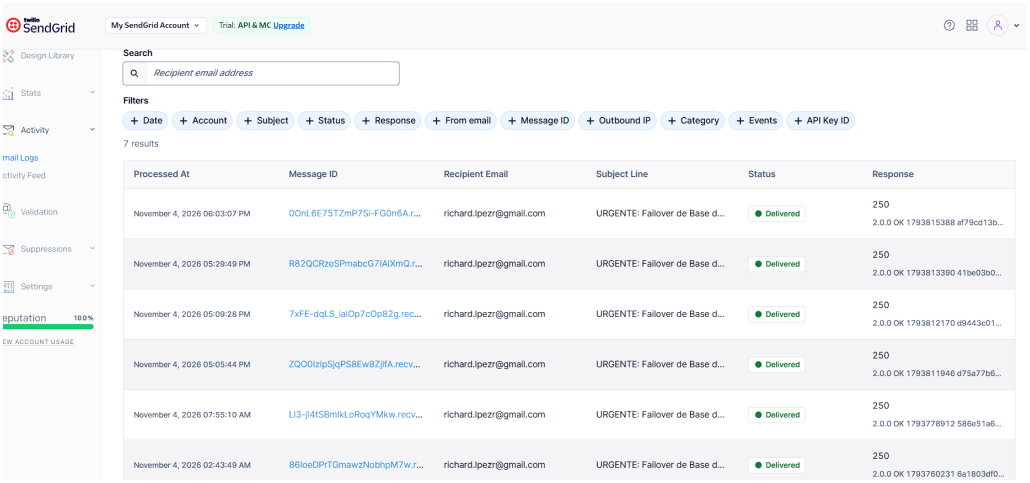


Figura 31: Web Services

20. Maquina Viertual GCP IaaS

La maquina virtual de Google Cloud. Dos instancias MySQL ejecutandose en Docker:

- mysql-master.
- mysql-slave.

Tabla 7: Almacenamiento GCP

Recurso	Capacidad
Disco raíz	24 GB
Espacio utilizado	22 GB
Espacio disponible	2.1 GB
Ocupación	92 %

```
richard97lopez@nodo-respaldo-ecobici:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        24G   22G  2.1G  92% /
tmpfs           477M     0  477M   0% /dev/shm
tmpfs           191M  1.4M  190M   1% /run
tmpfs           5.0M     0  5.0M   0% /run/lock
efivarfs        256K   32K  220K  13% /sys/firmware/efi/efivars
/dev/sda16      881M   52M  767M   7% /boot
/dev/sda15      105M  6.2M   99M   6% /boot/efi
tmpfs            96M  8.0K   96M   1% /run/user/1001
richard97lopez@nodo-respaldo-ecobici:~$
```

Figura 32: Web Services

20.1. RAM configurada por nodo

Tabla 8: Configuración del Buffer Pool en MySQL Master y Slave

Nodo	Buffer Pool (bytes)	Buffer Pool (MB)
MySQL Master	402,653,184	384 MB
MySQL Slave	402,653,184	384 MB
Total	**805,306,368**	**768 MB**

```
Your MySQL connection id is 16811
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
+-------------------------+-----------+
| Variable_name           | Value     |
+-------------------------+-----------+
| innodb_buffer_pool_size | 402653184 |
+-------------------------+-----------+
1 row in set (0.42 sec)

mysql>
```

Figura 33: Web Services

21. Consumo de RAM por contenedor

`docker stats -no-stream`.

Tabla 9: Consumo de memoria RAM por contenedor Docker

Contenedor	RAM utilizada	Límite mostrado	Consumo
mysql-haproxy	2.14 MiB	953.6 MiB	0.22 %
mysql-master	66.07 MiB	953.6 MiB	6.93 %
mysql-slave	41.55 MiB	953.6 MiB	4.36 %
Total	**109.76 MiB**	—	—

21.1. Distribución del consumo actual

El consumo de memoria RAM.

- **MySQL Master:** 66.07 MiB.
- **MySQL Slave:** 41.55 MiB.
- **HAProxy:** 2.14 MiB.

Tabla 10: Consumo total de memoria RAM

Concepto	Memoria utilizada
MySQL Master	66.07 MiB
MySQL Slave	41.55 MiB
HAProxy	2.14 MiB
Consumo total	**109.76 MiB**